

Adaptive Progressive Retrieval Attention: Interpretation, Search-Method Selection, Root-State Callbacks, and Bounded Retry

Jorge Simao

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 23, 2026

Abstract

Progressive Retrieval Attention (PRA) separates conceptual memory discovery from physical native-key/value (K/V) activation. This paper asks how much PRA effort a query should receive and when a bounded second attempt is justified. We first evaluate three coherent effort profiles, then open their coupling into independently selectable interpretation, search, and admission controls.

The factorized study replays frozen native graph and query scores for 95 validation and 65 held-out HotpotQA/QASPER example-seed units. It evaluates 792 valid configurations per unit over query facets F , roots R , successor breadth K , hop depth H , conceptual budget B_s , and physical K/V admission budget B_{kv} . On held-out HotpotQA, the factorized oracle preserves complete-chain quality 1.000 while reducing mean abstract cost from 34.18 for the cheapest sufficient E0/E1/E2 profile to 19.98. On the 24 QASPER units solved by both action spaces, cost falls from 16.11 to 8.95; factorization additionally solves one unit that no profile solves, yielding .833 versus .800 overall. Thus profile coupling hides substantial lower-cost combinations, although no learned router yet predicts those combinations reliably.

Five-seed factorized controllers make that limitation explicit. R0 profile routing obtains .631 chain quality at cost 24.29. Independent, conservative, signed-hash query, and autoregressive heads reduce cost to 16.13–18.06 but quality to .486–.560 because under-allocation rises. Parameter accuracy is therefore not the endpoint. A one-retry evaluator-side correction audit raises quality from .631 to the factorized ceiling .923, correcting all 15 initial HotpotQA failures and four of nine QASPER failures at 2.49 mean added cost; eight HotpotQA and QASPER cases use a single root/facet change, while compound fallback remains necessary elsewhere. This is retry headroom, not a learned online policy.

Normalized candidate accounting exposes a different controller tradeoff from the abstract cost alone. On QASPER, the factorized oracle recovers .917 of evidence parents while admitting 10.9% of candidates; targeted retry reaches the same recall at 19.0%. On HotpotQA, targeted retry raises evidence-parent recall from .733 to .948, but expands admission from 69.7% to 75.6%. These are identity-level discovery and native-K/V activation measurements, not generated-answer gains.

A separate four-dataset study treats retrieval representation itself as an action. It replays 132 frozen Paper-2.6 examples, of which 120 have a labelled successor decision. Validation selects exact root search, reaching .066 held-out root recall versus .104 for the per-example root oracle. For the 68 held-out examples with successor labels, a fixed BM25-root/native-semantic-successor pair reaches .030 recall; independently predicted root and successor methods reach .032, an interpretable cascade reaches .048, and the joint method oracle reaches .143. The method space therefore has real headroom, but the present small controller does not capture it. Learned targeted retry raises .032 only to .037, while an oracle correction reaches .143. These are matched-budget discovery metrics, not generated-answer or K/V-materialization gains.

We then strengthen request/reply with global, structural, multiscale, graph, and hierarchical structural-to-graph facets and execute the complete matched surface: 29,700 actions over the original 58 validation and 74 test identities, with five root and five successor methods. A validation-fixed global policy reaches .205 test evidence recall. Five-seed query-only one-shot control reaches $.180 \pm .022$ without graph and $.180 \pm .023$ with graph. A learned single callback after root selection falls to .168; a validation-tuned uncertainty rule reaches $.188 \pm .024$ while invoking graph refinement on 16.5% of cases. Complete-action and stage-wise evaluator oracles both reach .365 exactly. Their equality is important: on this measured Cartesian surface, observing root state creates no additional oracle action, and learned callbacks do not justify serving complexity. A stratified eight-example frozen Qwen decode check maps four conceptual selections to 128 native K/V tokens across eight consumer layers. One-shot and threshold policies raise exact match/token F1 from .375 without memory to .500; the callback result is .375. This small confirmation verifies materialization and generation but is not a benchmark-quality answer comparison.

Query interpretation is itself an action. Explicit and structural regions remove recent-head serialization sensitivity in matched controls and remain stable through 8K following tokens. Structural selection is perfect on six semi-natural fixtures but falls to .400 under five deliberate role decoys, while the competing recent-head interpretation remains correct there. The result motivates disagreement-aware reinterpretation rather than unconditional structural replacement. We also ask whether the frozen Qwen3-0.6B backbone should read its own question before allocating PRA effort. Four-fold grouped validation selects the last question-token embedding, but its five-seed held-out quality is .668 at total cost 27.20, below the observable profile router’s .702 at 28.25. The validation-selected source-plus-query upper bound reaches only .674 while paying a 256-token prefill. Thus model-native query state does not yet justify replacing observable R0 or introducing an external encoder. Exact prefix continuation does, however, show that representations captured before the first PRA consumer can be reused without re-computing the query. All primary quality results are frozen-score evidence-chain endpoints, not generated-answer scores. Kernel, cache, batching, and serving-engine optimization are handed to Papers 5.5 and 6.

1 Introduction

A fixed memory policy is wrong in two directions. A narrow policy wastes quality on questions whose evidence is dispersed or connected by several relations. A broad policy wastes search, transfer, K/V capacity, and attention competition on questions that can be answered from one compact region. The relevant systems question is therefore not only which memory should be retrieved, but how much effort the current query justifies.

PRA provides several independent control surfaces. A runtime may form one or several query facets, retain one or several roots, expand different numbers of neighbors and hops, search at different layers, and materialize anything from a compact evidence interval to a broader source region. Paper 3 showed that conceptual breadth and physical disclosure are not the same budget: once identity is fixed, exact native-K/V evidence can outperform a whole parent while using far fewer physical states. The present work turns those controls into an adaptive inference contract.

The central test is

$$Q_{\text{adaptive}} \approx Q_{\text{always-high}} \quad \text{while} \quad \mathbb{E}[C_{\text{adaptive}}] < C_{\text{always-high}}, \quad (1)$$

where Q is held-out output or retrieval quality and C includes conceptual search, physical K/V, generation retries, and measured latency. Equation 1 is not guaranteed on every dataset. If almost every query is hard, the correct controller should spend the high budget.

This paper makes thirteen contributions:

1. It defines a public PRA control vector and freezes three monotonic effort profiles rather than predicting unconstrained continuous parameters.
2. It implements hand-rule and learned minimum-effort controllers using only observable query, routing, output, and memory features; evaluator labels are rejected by the serving API.
3. It provides manual and automatic retry modes with incremental state reuse, hard budgets, semantic answer consistency, and structured decision traces.
4. It evaluates validation-frozen adaptive compute, calibration, retry recovery, and oracle minimum effort on held-out HotpotQA and QASPER traces.
5. It makes query-region location and extent explicit, supports structured prompt roles, and tests matched query-first, query-last, query-middle, log, and URL-list layouts.
6. It measures profile quantization regret over 792 valid factorized configurations per unit, including token-weighted evidence precision and recall and separate search/admission budgets.
7. It retrains profile, independent-head, signed-hash query, conservative, and autoregressive routers on validation-derived factorized targets rather than inherited profile labels.
8. It evaluates bounded single-control correction and compound fallback as explicit retry upper bounds, recording wrong-to-correct, correct-to-broken, extra work, and action identity.
9. It compares signed hashing, token embeddings, early/middle/final query-only states, native Q/K geometry, and an expensive contextual upper bound under grouped validation selection.
10. It tests coherent group profiles against global, independent, and autoregressive factorized targets, and supplies an exact pre-consumer query-prefill reuse path with cost accounting.
11. It makes root and successor retrieval representations independent controller actions, imports the Paper-2.6 action contract by content hash, and compares fixed, cascade, learned, and oracle method policies on four natural evidence tasks.
12. It integrates tensor-native semantic query graphs into the same one-shot request/reply action, adds hierarchical syntactic-to-graph facets and per-facet channel policies, and reports graph cost, overlap, five-seed controller, and oracle-headroom measurements.
13. It implements one opt-in ROOT_SELECTED callback, executes a matched 29,700-action root/successor surface, compares learned and threshold controls under standard and leave-one-dataset-out splits, and confirms selected identities through native-K/V materialization and frozen decode.

Full-weight PRA retraining is deliberately absent. That intervention belongs to Paper 4. Here the scientific object is adaptive inference over frozen routing and consumer behavior.

2 Adaptive PRA as a Control Problem

2.1 Control vector

At attempt t , the runtime selects

$$a_t = (\mathcal{Q}, F_m, P_c, \theta_g, S_r, S_s, F, R, K, H, B_s, B_{kv}, \theta, L_s, L_c, G, M), \quad (2)$$

where $\mathcal{Q}$ is one or more functional query regions, S_r and S_s are the root and successor search representations, F is the facet policy, R retained roots, K candidate neighbors per expansion, H hop depth, B_s conceptual search budget, B_{kv} physical native-K/V budget, θ a closure threshold, L_s search layers, L_c consumer layers, G routing/materialization granularity, and M disclosure policy. We separate B_s and B_{kv} because Paper 3 found that broad search can remain compatible with compact evidence-only activation. We also do not force $S_r = S_s$: an admitted chunk changes the successor query state and can expose an address best resolved in another representation. Here $F_m \in \{\text{global, syntactic, multiscale, graph, syntactic} \rightarrow \text{graph}\}$ is the facet-construction mode, P_c is the coarse partition policy, and θ_g denotes the query-graph similarity, threshold, top- k , and component constraints. Keeping F_m separate from facet count prevents a controller from treating “four windows” and “four graph communities” as the same interpretation action.

The first public implementation uses three profiles (Table 1). They are not universal hyperparameters; they are a stable SDK contract and an initial experimental ladder.

Table 1: Validation-frozen effort profiles. The layer identifiers refer to the inherited 28-layer Qwen experiments. Lower routing thresholds permit broader closure.

Profile	$\mathcal{Q}$	facets	R	K	H	B_c	B_{kv}	G	consumer layers
E0 low	head	1	1	2	0	2	256	256	27
E1 medium	structural	2	2	4	1	4	512	128	26–27
E2 high	multi-region	4	4	8	3	8	1024	64	24–27

Escalation is monotonic in permitted facets, roots, neighbors, hops, conceptual budget, K/V budget, and layer sets. A lower θ is broader. Code rejects a ladder in which a later profile removes a prior resource. Monotonic permission does not imply monotonic answer quality: extra memory may dilute attention, so retries must also record correct-to-incorrect transitions.

2.2 Observable state and leakage boundary

The controller may observe cheap query statistics, score gaps, routing entropy, root competition, facet agreement, frontier shape, newly discovered memory, path convergence, output entropy, answer log probability and margin, retry consistency, active K/V, memory-attention ratio, and a relevance-density proxy. Ground-truth answers, oracle identities, gold evidence, and evidence recall are evaluator fields. They cannot enter the serving feature constructor.

For vocabulary distribution p_t , token entropy is

$$H_t = - \sum_v p_t(v) \log p_t(v), \quad \bar{H} = \frac{1}{T} \sum_{t=1}^T H_t. \quad (3)$$

Entropy is one signal. A fluent wrong answer can have low entropy. We therefore compare entropy-only, answer-margin, routing-only, and combined predictors rather than defining confidence as a single scalar by fiat.

2.3 Minimum-effort prediction

For validation example i , let

$$e_i^* = \min\{e \in \{E0, E1, E2\} : Q_i(e) \geq q_i^*\}, \quad (4)$$

where q_i^* is the target quality criterion. If no profile succeeds, $E2$ is retained as the bounded fallback. The oracle e_i^* is used only to train or analyze the controller. At test time, a standardized ridge classifier predicts one profile from the initial observable feature vector. The classifier is deliberately small, deterministic, and serializable without a new runtime dependency.

Controller regret is

$$\text{regret}_i = C(\hat{e}_i) - C(e_i^*) \quad (5)$$

subject to reporting whether $Q_i(\hat{e}_i)$ matches $Q_i(e_i^*)$. Negative regret without matched quality is not a saving.

2.4 Query-region discovery: separating intent from position

The runtime first asks which prompt region specifies the retrieval objective and only then asks how to facet that region. These are distinct decisions:

$$\text{prompt} \rightarrow \mathcal{Q} = \{[s_j, e_j]\}_j \rightarrow (\{q_1, \dots, q_F\}, S_r) \rightarrow \text{roots} \rightarrow (S_s, \text{search}) \rightarrow \text{admit} \rightarrow \text{evaluate}. \quad (6)$$

Functional roles include instruction, query, current state, context, references, history, and tool or log output. Position does not determine role. Explicit token spans or caller-labeled segments are authoritative. Raw text falls back to a deterministic structural ladder that recognizes question/instruction headings and punctuation while discounting code fences, logs, context labels, and URL lists. A retry may reinterpret a recent-head cue as a structural or multi-region cue before increasing R , K , H , or K/V budget.

The selection records start, end, region count, method, confidence, and score margin. Multiple regions support a persistent instruction plus a narrower current question; session mode selects the smallest current query state rather than the whole conversation. The heuristic is deliberately a baseline. A learned span scorer is justified only if explicit-span headroom remains after structural selection on natural prompts.

2.5 Learned effort routing: profiles to interacting heads

All controller families target minimum sufficient effort rather than the highest-scoring action:

$$a_i^* = \arg \min_a C(a) \quad \text{subject to} \quad Q_i(a) \geq q_i^*. \quad (7)$$

R0 selects one validation-frozen profile. R1 uses a shared feature MLP and independent categorical heads. R2 adds a cached query-region representation. R3A predicts controls autoregressively in the order $\mathcal{Q} \rightarrow S_r \rightarrow F \rightarrow R \rightarrow S_s \rightarrow K \rightarrow H \rightarrow B_s \rightarrow B_{kv}$. The method heads are added only as measured categorical targets; interaction-aware expansion remains gated by Section 8. Integer controls and thresholds use finite categorical values; no Cartesian action class is constructed. The first comparison inherits profile-derived labels; the decisive comparison is retrained later on independently measured factorized-oracle targets. For independent heads,

$$\mathcal{L} = \sum_j \lambda_j \text{CE}(\hat{a}_j, a_j^*). \quad (8)$$

Evaluation reports each head, under- and over-allocation, quality regret, cost regret, calibration, and latency. Joint exact match is secondary to the held-out quality/expected-effort frontier.

3 Retry and Runtime Design

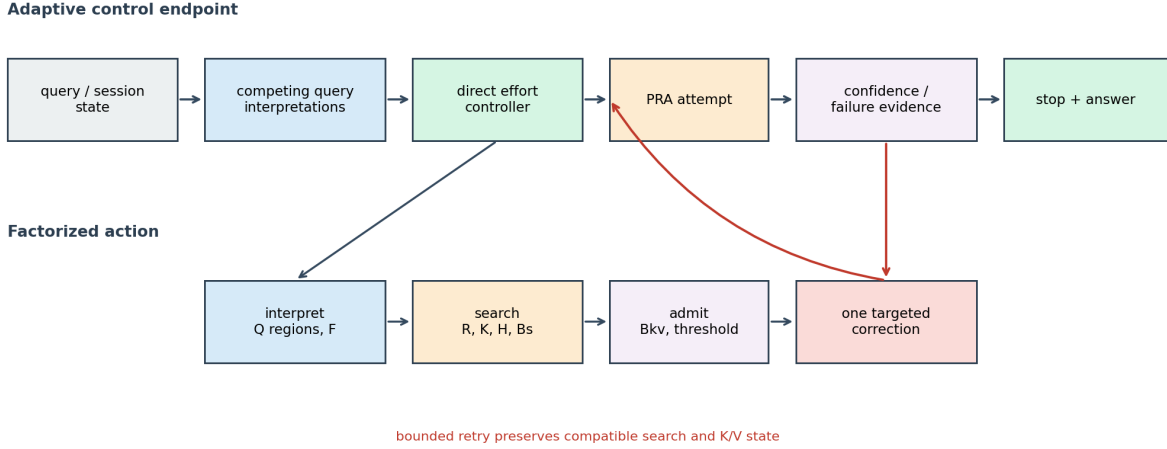


Figure 1: Adaptive PRA control endpoint. Competing query interpretations feed a direct profile or factorized action. The extended action also selects independent root and successor search methods. Observable failure evidence may trigger one targeted correction with compatible search and K/V reuse; otherwise the controller stops.

3.1 State machine

Attempt zero uses the selected profile or factorized action. A manual caller may name low, medium, or high effort, provide individual controls, and request one bounded retry. Automatic mode applies a calibrated stop rule. If it fails, the corrective action may reinterpret the query, widen facets, change root or successor representation, increase roots, neighbors, hops, or search budget, or change K/V admission while preserving compatible roots, search results, K/V state, and prefix state. Search budget, active K/V, retry count, and latency are hard caps.

The full controller cycle is therefore

QUERY/SESSION → INTERPRET → SEARCH → ADMIT → EVALUATE → targeted retry.

The public transition is:

```

profile = controller.choose(observable_features)
while True:
    result = execute(profile, reusable_state=previous)
    stop, signals = stop_policy(result.features, result.error_probability)
    trace.append(public_decision(result, profile, signals))
    if stop or budget_exhausted(profile, trace):
        return result, trace
    profile = targeted_correction(profile, observable_failure_signals)
    previous = result

```

Answers across attempts are compared with normalized token F1. This is a cheap semantic proxy, not a full entailment judge. The runtime records initially-wrong-to-correct, initially-correct-to-wrong, and no-change retries separately.

3.2 Structured traces

A trace contains attempt index, profile and control vector, active K/V, selected-parent count, incorrect-answer probability, routing entropy, answer margin, consistency, search/materialization/generation time, reuse counts, query-region bounds, confidence, method and expansion count, escalation signals, and stop reason. It does not contain hidden chain-of-thought. Evaluator outcomes live in experiment rows outside the controller feature path.

3.3 Control/execution boundary

Let $Q \in \mathbb{R}^{n_q \times d}$ be native query states and $K_m \in \mathbb{R}^{n_m \times d}$ indexed memory keys. Exact batched search computes

$$S = QK_m^\top, \quad I = \text{TopK}(S, k). \quad (9)$$

This equation defines the search work exposed to the controller. How a later runtime compiles or indexes it is outside the adaptive-policy contribution; approximation is acceptable only under a separate matched-recall systems gate.

3.4 Search/admission boundary

Conceptual search returns source-parent identities. Admission maps a bounded subset to native K/V with shape $[B, H, T, D_h]$. The adaptive trace must record both the searched parent budget and the physically admitted token count. PagedAttention controls where K/V is stored [5]; PRA control decides which logical memory becomes active. Physical layout optimization is deferred.

4 Experimental Protocol

4.1 Adaptive-compute cohort

The controller study reuses frozen Paper-2.5 semantic exploration traces. The selected root policy is seed agreement at .6; transition policy is fixed Top-4. Fractions .1, .2, and .4 instantiate E0, E1, and E2. There are 95 validation and 65 held-out example-seed units across HotpotQA and QASPER. Validation chooses the hand rule, learned classifier, failure predictor, and retry threshold. No threshold is changed after held-out evaluation.

The quality endpoint is complete evidence-chain recovery. This is a routing endpoint, not generated answer accuracy. Output confidence is therefore evaluated separately with Paper 3’s controlled model, using its validation/held-out split and actual prediction entropy, NLL, answer margin, and correctness. Keeping the two cohorts separate avoids fabricating a joint output/routing signal that was not measured.

Abstract effort is deterministic:

$$C(e) = RK(H + 1) + B_c + B_{kv}/64. \quad (10)$$

A retry pays the final monotonic profile plus a fixed controlled regeneration charge of two units per additional attempt. Measured routing latency is reported separately.

4.2 Factorized-control cohort

The same 95/65 grouped split is replayed from frozen Qwen native adjacency, query-facet scores, source spans, and evidence groups. Validation supplies factorized router targets; held-out targets

are evaluator-only. The full valid lattice is executed on both partitions so the held-out oracle is an evaluation ceiling, not a pruning input. Pareto dominance maximizes complete-chain quality, evidence recall, and evidence precision while minimizing abstract cost, active K/V, and transition comparisons.

R0 is trained against the minimum sufficient profile. R1, R2, and R3A are trained against the minimum-cost factorized action. R2’s 32-dimensional signed-hash input uses the actual question text. All neural controllers use five fixed initialization seeds. The retry audit begins from one frozen R0 seed and permits one single-control correction or a separately labeled compound fallback.

4.3 Self-encoded query-router cohort

The self-router add-on keeps the factorized surface, identities, partitions, and five controller seeds fixed. It freezes Qwen3-0.6B at revision `c1899de289a04d12100db370d81485cdf75e47ca`. For each of the 32 unique questions, the normal question-only chat prompt is encoded with memory disabled; pooling is restricted to the explicit question span. The sweep includes token embeddings, hidden states after layers 4, 7, 14, 18, and 28, and native Q/K states after Qwen’s per-head RMS normalization and before RoPE at representative layers. Mean pooling is primary; last token is tested only for embeddings and layers 14 and 28. A 256-token ordinary source-plus-question prompt supplies a contextual diagnostic upper bound.

Every high-dimensional representation is compressed by a 16-dimensional PCA fitted only on unique validation identities. Four-fold validation cross-validation, grouped by example identity, selects representations and compact codebooks; the 65 held-out example-seed units do not participate. The same two-layer 32-unit profile head and five initialization seeds are used across sources. The target-granularity gate compares validation-derived global profiles, coherent interpret/search/admit groups, independent categorical values, and the sequence $F \rightarrow R \rightarrow K \rightarrow H \rightarrow B$.

A separate query prepass through depth k is charged

$$C_q = \frac{T_q \max(1, k)}{64L}, \quad C_{\text{total}} = C(a) + C_q, \quad (11)$$

where T_q includes the question chat wrapper and $L = 28$. We also report cumulative warmed CUDA event latency on a GTX 950M. If capture stops before the first possible PRA consumer (layer 24), the exact intermediate state may be reused and the additional C_q is zero. Tests require bitwise parity with uninterrupted Qwen execution and verify that the prepass mutates no parameter or cache.

4.4 Inherited measurement scope

Search, gather, page, cache, and batching benchmarks run on CPU with PyTorch 2.12.1. Exact search uses 32 queries, 64-dimensional keys, Top-8, and 256–4,096 memories. Kernel tests use 8 heads and head dimension 32. Page tests store 1,537 tokens and select 128 clustered or dispersed tokens. Batch tests use 1–32 heterogeneous requests. We report medians and mark HBM estimates as extrapolated.

These earlier CPU components and held-out Qwen observations are retained only as provenance for the Paper-5.5 handoff. They are not used to select factorized actions and are not adaptive end-to-end serving measurements.

4.5 Inherited baseline discipline

The matched synthetic corpus contains 1,024 documents and 200 two-hop queries. The query retrieves a bridge; the answer evidence is linked from that bridge. Native context, truncation, one-shot RAG, multi-query RAG, iterative RAG, fixed PRA, and adaptive PRA share the corpus and retrieval geometry. This controlled construction tests cost accounting and the need for iteration. It does not replace a full production RAG evaluation [6, 13].

H2O [17], SnapKV [7], StreamingLLM [15], and ClusterKV [8] are represented by mechanism-faithful controlled selectors at matched active-K/V budgets. No upstream kernel is claimed. Sparse architectural methods are discussed rather than experimentally mixed with incompatible checkpoints.

5 Adaptive-Compute Results

5.1 Oracle ceiling and direct control

The oracle audit is strongly dataset-dependent. On HotpotQA, 31 of 35 held-out units require E2, two require E1, and two require E0. On QASPER, 24 of 30 require only E0 and six require E2. This heterogeneity creates a large QASPER adaptive-compute opportunity but almost none on HotpotQA.

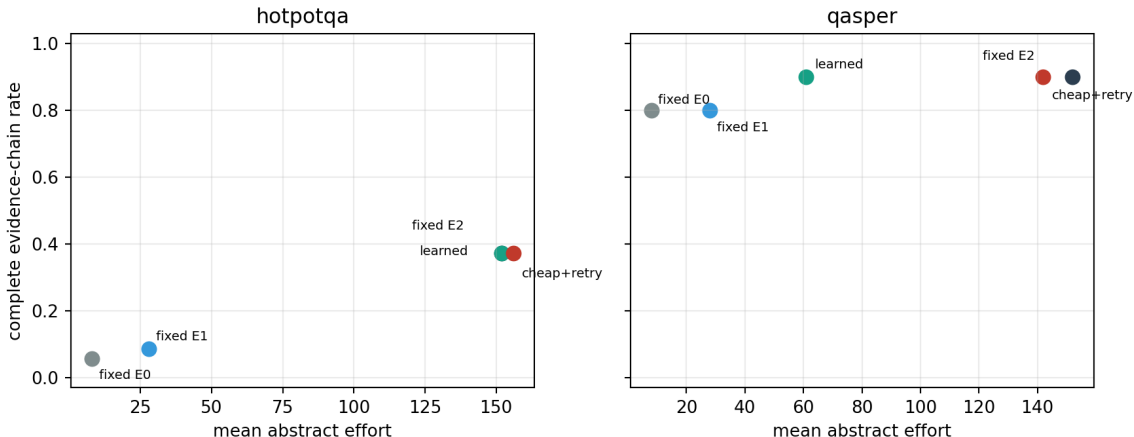


Figure 2: Held-out complete-chain quality versus mean abstract effort. Learned control matches E2 on both datasets. It saves effort on QASPER but not on the predominantly hard HotpotQA cohort.

Table 2: Held-out adaptive-compute frontier. Cost includes conceptual search and physical K/V; retry rows add the controlled regeneration charge.

Dataset	Policy	chain quality	mean cost	attempts	escalation
HotpotQA	fixed E0	.057	8.0	1.00	.000
HotpotQA	fixed E2	.371	152.0	1.00	.000
HotpotQA	learned direct	.371	152.0	1.00	.000
HotpotQA	cheap + retry	.371	156.0	3.00	1.000
QASPER	fixed E0	.800	8.0	1.00	.000
QASPER	fixed E2	.900	152.0	1.00	.000
QASPER	learned direct	.900	60.8	1.00	.000
QASPER	cheap + retry	.900	141.9	2.83	.933

The learned direct policy reduces QASPER mean effort by 60.0% with no chain-quality loss. Its mean oracle cost regret is 24 units and quality match is 1.0, so further gains remain possible. On HotpotQA, the learned and hand controllers both match oracle-profile quality but incur positive regret because a few easy cases are conservatively sent to E2.

5.2 Retry recovery is not free

Cheap-first retry corrects 31.4% of HotpotQA units that fail E0 and 10.0% of QASPER units. It does not break an initially correct unit in this trace. However, HotpotQA reaches E2 on every request and pays two extra generation attempts, raising cost from 152.0 to 156.0. QASPER stops early on two units but still escalates 93.3%, reducing cost only to 141.9. Retry is valuable as a recovery and fallback mechanism; the learned initial profile is the main source of QASPER savings.

This result also distinguishes routing reuse from generation reuse. Roots and K/V states can be preserved incrementally, but a changed answer may still require regeneration. A serving claim must measure discarded tokens and prefix-cache reuse rather than treating monotonic search as free.

5.3 Confidence and selective risk

Table 3: Held-out incorrect/failure prediction. Output rows use the controlled Paper-3 model; routing rows use frozen Paper-2.5 traces.

Cohort	Signals	AUROC	AUPRC	ECE	Brier
Output	entropy only	.403	.899	.229	.145
Output	answer margin only	1.000	1.000	.277	.133
Output	combined output	1.000	1.000	.291	.117
Routing	routing entropy only	.652	.690	.043	.249
Routing	root gaps	.802	.816	.117	.197
Routing	combined runtime	.824	.853	.139	.187

The output cohort is small (40 held-out observations, 90% incorrect), so perfect rank separation for answer margin is diagnostic rather than a deployment estimate. Entropy-only ranking is worse than random in this cohort despite high AUPRC caused by class imbalance. No incorrect output is assigned less than .2 error probability by the fitted predictors, but this does not establish that confidently wrong answers are absent in broader language tasks.

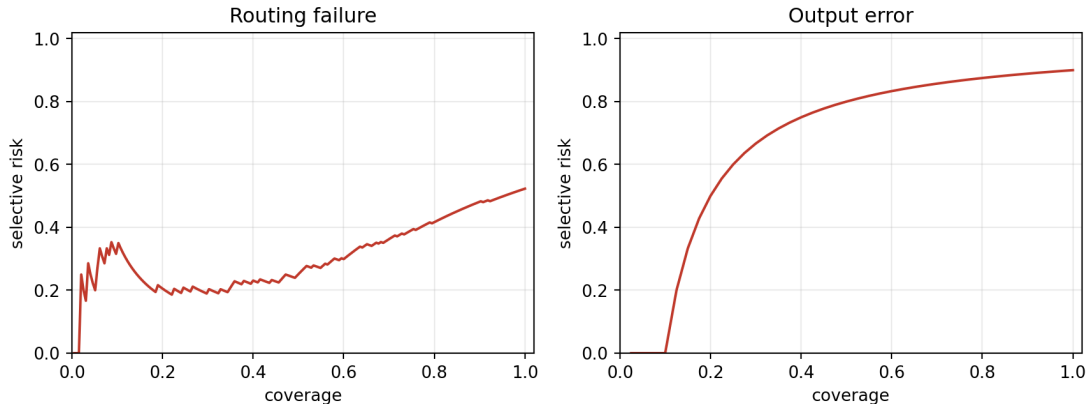


Figure 3: Selective risk as increasingly uncertain observations are accepted. Curves are reported separately because routing and output confidence were not observed on one unified live cohort.

6 Query-Region and Router-Architecture Results

6.1 Matched prompt layouts

The controlled corpus crosses 24 semantic cases, three payload forms (prose, logs, and URL lists), and five serializations. Each row keeps evidence, eight candidate roots, one selected root, 16 active native-K/V tokens, and a global facet fixed. The lexical candidate geometry is intentionally simple: the experiment isolates cue position and role detection, not pretrained semantic routing.

Table 4: Matched query-layout headline result. Each policy cell is root R@1 / answer-quality proxy; active K/V is 16 throughout. Effort lists head / explicit / structural / reinterpretation lexical comparisons.

Layout	head	explicit	structural	head+reinterpret	active K/V	effort
$C + Q$	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00	16	8/8/8/8
$Q + C$	.00/.00	1.00/1.00	1.00/1.00	1.00/1.00	16	8/8/8/16
$C_1 + Q + C_2$	.00/.00	1.00/1.00	1.00/1.00	1.00/1.00	16	8/8/8/16
$I + C + Q$	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00	16	8/8/8/8
Q +long payload	.00/.00	1.00/1.00	1.00/1.00	1.00/1.00	16	8/8/8/16

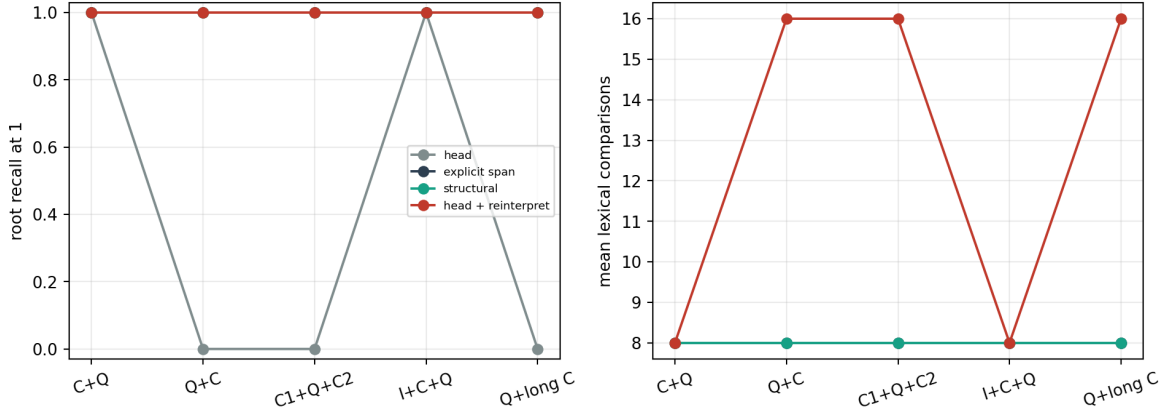


Figure 4: Serialization changes a suffix-head cue but not explicit or structurally discovered regions in the matched control. Reinterpretation pays a second search only when the structural cue does not overlap the recent head.

Head R@1 averages .400 and its layout spread is 1.0. Explicit and structural regions both average 1.000 with zero spread, so Gate 1 and Gate 2 pass in this corpus. Reinterpretation corrects all 216 head failures, breaks none of 144 initially correct cases, and adds eight comparisons per corrected case (4.8 averaged over all requests). This perfect heuristic result follows from explicit question labels and punctuation; it is a structural control, not evidence that natural query-role discovery is solved.

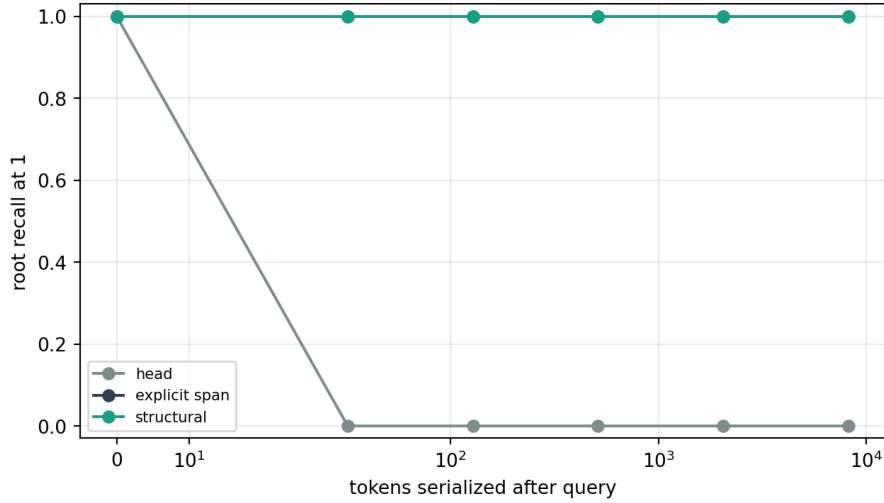


Figure 5: Early-query displacement. The 16-token head fails once payload follows the query, while explicit and structural regions preserve root R@1 through 8,192 following payload tokens.

The displacement sweep makes the motivating failure direct: head R@1 changes from 1.0 at zero following tokens to zero at 32 and remains zero through 8K; explicit and structural selection remain 1.0. Multi-region, facet, root-count, URL-role, and session-isolation rows are retained in the public artifacts. Broad all-region selection reaches only .333 overall, so more query-region breadth is not monotonically better.

6.2 Semi-natural and adversarial role fixtures

Six additional fixtures place the operative question inside email, tool output, source code, logs, URL lists, and multi-turn conversation. Structural discovery obtains root R@1 1.000 and mean span IoU .981; explicit spans obtain 1.000 on both measures. These remain constructed fixtures, but their surface form is less regular than the original matched template.

Five adversarial fixtures insert a quoted previous question, a question-like log field, a misleading heading, a code-comment question, or a URL containing query-like text. Structural R@1 falls to .400 and span IoU to .314. Recent-head remains correct on all five because the current question is later than the decoy. The bounded policy therefore keeps the recent interpretation rather than replacing it merely because a structural cue exists. This negative control changes the design conclusion: structural discovery is one candidate interpretation, and disagreement or a small score gap should trigger comparison, not unconditional trust.

6.3 Preliminary profile-derived router capacity

R0–R3A share the 95 validation / 65 test split and validation-derived minimum-effort labels. R1 uses runtime features only. R2 adds a 32-dimensional signed-hashing query channel to exercise the semantic-router interface without downloading or claiming a pretrained encoder. R3A adds autoregressive parameter conditioning. R1–R3A use five seeds.

Table 5: Held-out router architecture frontier. Quality is complete-chain rate; cost is abstract PRA effort. R0 has one deterministic fit; learned multi-head values pool five seeds.

Router	quality	cost	joint exact	mean head acc.	under	over
R0 profile	.615	109.9	.800	.800	.031	.169
R1 feature MLP	.594	103.7	.775	.779	.055	.166
R2 semantic MLP	.591	104.0	.794	.799	.046	.157
R3A autoregressive	.591	102.6	.806	.806	.049	.145

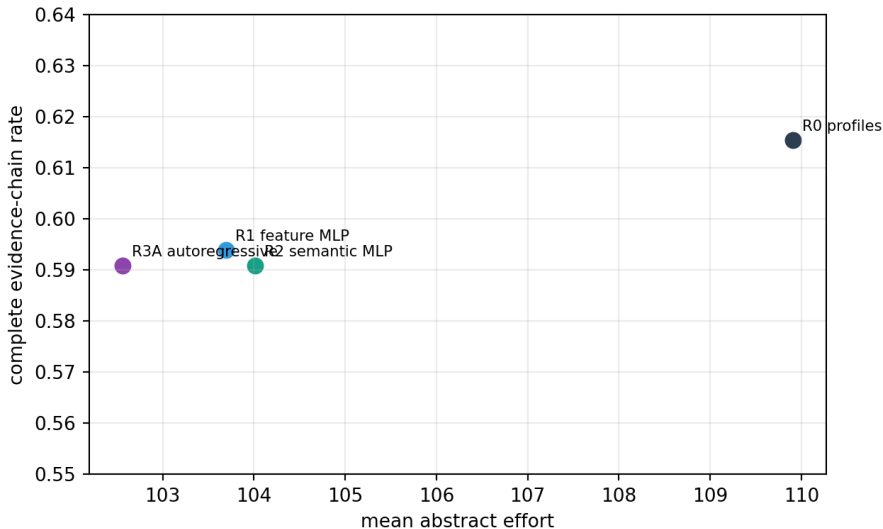


Figure 6: Added controller capacity does not improve the held-out quality/cost frontier. R3A is slightly cheaper than R2 at the same quality, but both remain below R0 quality.

The profile selector remains the supported controller. R1–R3A save 5.9–7.3 cost units relative to R0 by under-allocating more often, but lose 2.2–2.5 quality points. R3A improves joint parameter exactness to .806 and lowers cost relative to R2 without restoring quality. The interaction gate therefore fails: no controller transformer is built. Parameter mutual information is reported only as profile-label coupling because all heads inherit three jointly defined profiles; it is not causal evidence that changing one PRA control changes another.

7 Factorized Control Results

7.1 Measured action lattice and cost

The preliminary multi-head comparison cannot answer whether profiles are the right action unit: every target field was copied from E0, E1, or E2. We therefore replay the frozen native adjacency and query-facet scores over

$$(F, R, K, H, B_s, B_{kv}) \in \{1, 2, 4\} \times \{1, 2, 4\} \times \{2, 4, 8\} \times \{0, 1, 2, 3\} \times \{2, 4, 8\} \times \{2, 4, 8\}. \quad (12)$$

Configurations with $B_s < R$ or $B_{kv} < R$ are invalid, leaving 792 measured actions per example–seed unit. Query-region policy is fixed to structural, native edge admission is open, terminal-goal stopping is closed, and best-first traversal is fixed. These controls isolate profile coupling rather than reopening every Paper-2.5 routing choice.

For each execution, root scoring compares F facets with all source parents; graph search records actual native proposals; and physical admission preserves roots before admitting discovered parents in search order. The normalized accounting used only for within-study ordering is

$$C(a) = F + N_{\text{transition}} + B_s + \frac{T_{kv}}{64}, \quad (13)$$

where T_{kv} is the number of materialized native-K/V source tokens. The artifacts also retain unscaled root comparisons FN_{parents} , transition comparisons, conceptual parents, K/V tokens, and wall time. Thus Equation 13 is auditable rather than a latency surrogate.

Physical evidence metrics are token weighted:

$$P_{E,kv} = \frac{\text{materialized evidence K/V tokens}}{\text{all materialized K/V tokens}}, \quad R_{E,kv} = \frac{\text{materialized evidence K/V tokens}}{\text{required evidence K/V tokens}}. \quad (14)$$

Complete-chain quality requires every evidence group to be represented after physical admission. No generated answer exists for these newly factorized configurations, so no output oracle is imputed.

7.2 Profile quantization regret

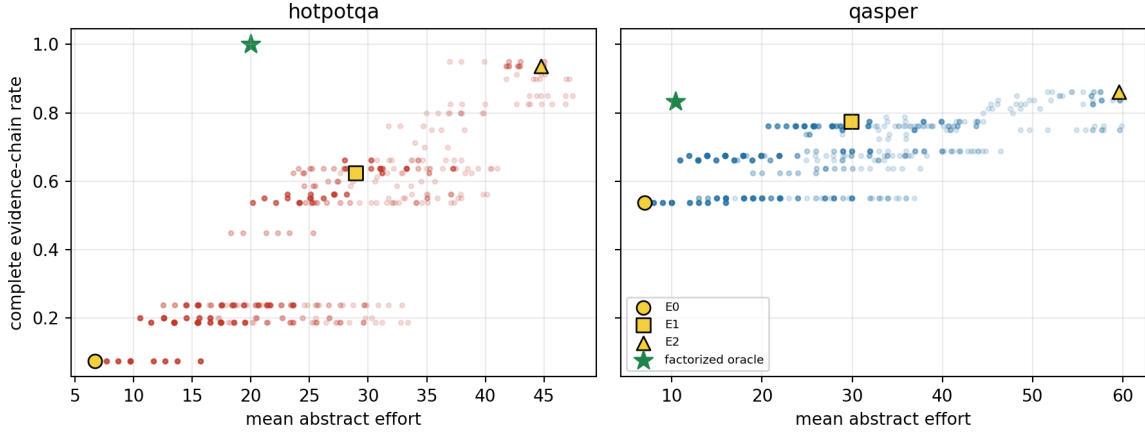


Figure 7: Held-out factorized quality–cost surface. Stars are per-example factorized oracles averaged by dataset; E0–E2 are diagonal profile configurations under the same accounting.

Table 6: Held-out profile quantization audit. Matched rows include units solved by both action spaces. QASPER overall quality is also shown because factorization solves one additional unit.

Dataset	matched n	profile quality	factor quality	profile cost	factor cost	regret
HotpotQA	35	1.000	1.000	34.18	19.98	14.20
QASPER matched	24	1.000	1.000	16.11	8.95	7.15
QASPER all	30	.800	.833	16.82	10.43	—

HotpotQA supports the profile-aliasing hypothesis rather than the intrinsically-high-effort hypothesis: difficult examples often need one expensive dimension, not the full E2 bundle. Mean cost falls 41.5% at matched complete-chain quality. On matched QASPER units the aggregate reduction is 44.4%. The absolute costs should not be compared with the earlier 8/28/152 convention; Table 6 recomputes both spaces with Equation 13.

7.3 What the oracle actually requests

Table 7: Held-out minimum-cost factorized-oracle distributions. Entries are percentages of example–seed units; omitted levels have zero mass.

Control	HotpotQA	QASPER
F	1: 97.1, 4: 2.9	1: 96.7, 2: 3.3
R	1: 65.7, 2: 8.6, 4: 25.7	1: 66.7, 2: 23.3, 4: 10.0
K	2: 80.0, 4: 5.7, 8: 14.3	2: 96.7, 4: 3.3
H	0: 45.7, 1: 54.3	0: 96.7, 1: 3.3
B_s	2: 34.3, 4: 45.7, 8: 20.0	2: 86.7, 4: 10.0, 8: 3.3
B_{kv}	2: 34.3, 4: 45.7, 8: 20.0	2: 86.7, 4: 10.0, 8: 3.3

Nearly every unit uses one facet and low successor breadth. HotpotQA’s extra work is concentrated in roots, one graph hop, and larger parent budgets; QASPER is predominantly root-only. No held-

out oracle chooses $B_s > B_{kv}$. Broader-search/narrower-admission configurations are measured, but under deterministic search-order admission they do not become the cheapest complete physical chain. Separating the budgets remains architecturally necessary; this admission rule does not yet exploit it.

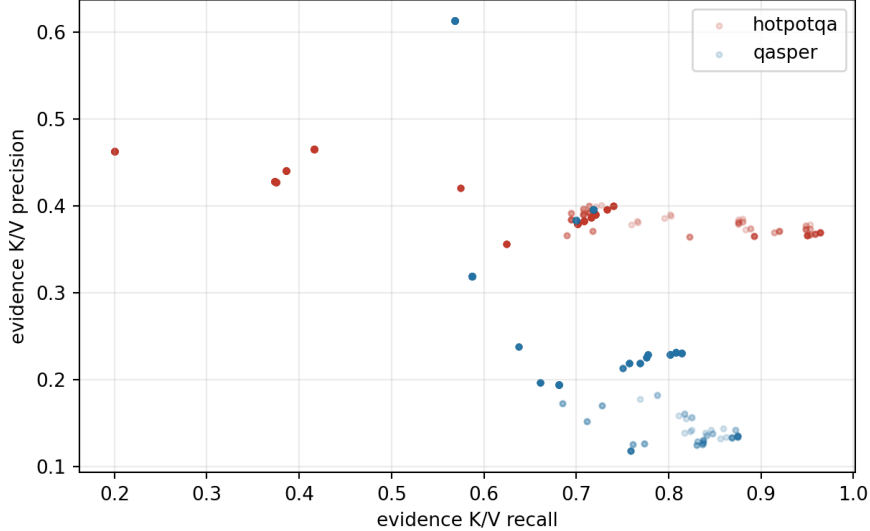


Figure 8: Token-weighted physical evidence precision and recall across aggregate factorized configurations. Broad conceptual recovery does not guarantee a precise admitted K/V set.

7.4 Routers retrained on factorized targets

Every learned router is trained only on 95 validation units and evaluated on 65 test units over five initialization seeds. R2 now hashes actual question text rather than a dataset-class phrase. Conservative R1/R2 decoding selects an upper categorical quantile. Invalid independently predicted budgets are raised only enough to preserve all roots, and the repair rate remains below 2.2%.

Table 8: Five-seed held-out router frontier on factorized targets. Under-allocation means failing a chain the factorized oracle completes; over-allocation means matching quality at higher cost.

Router	quality	cost	cost regret	under	over
R0 profile	.631	24.29	8.72	.292	.523
R1 independent	.538	16.13	.56	.385	.378
R1 conservative	.560	17.97	2.40	.363	.422
R2 query hash	.505	16.55	.98	.418	.335
R2 conservative	.548	18.06	2.48	.375	.400
R3A autoregressive	.486	17.18	1.61	.437	.335

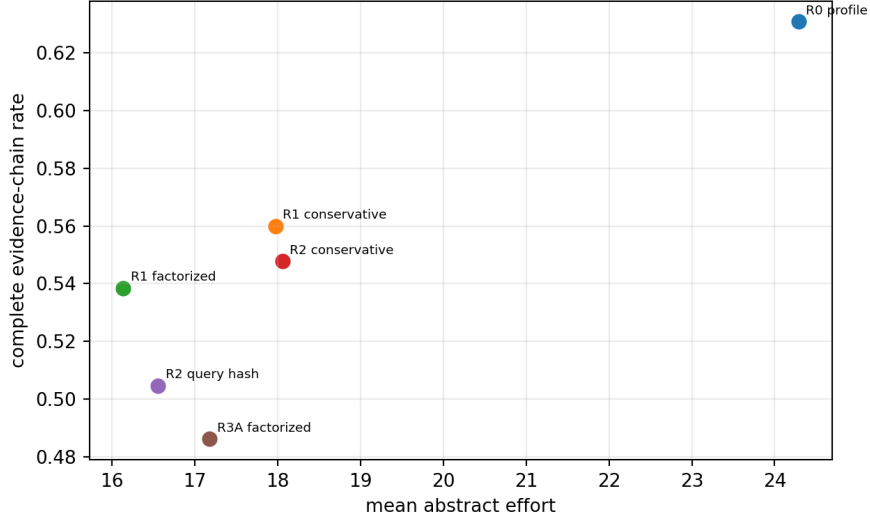


Figure 9: Factorized controller quality versus expected cost. Lower nominal cost is obtained by missing sufficient actions more often; no factorized learned router dominates R0.

True factorized labels prove that cheaper actions exist, but the small cohort and signed-hash channel do not predict them safely. Conservative decoding recovers 2.2–4.3 quality points over its argmax counterpart, still below R0. Section 10 tests model-native semantic channels before any external encoder; a larger grouped natural dataset remains necessary.

7.5 Bounded targeted-retry headroom

We take one frozen-seed R0 decision as the initial action and permit one corrective attempt. The audit first chooses the cheapest successful action differing in exactly one control. If none exists, it records a compound fallback to the factorized oracle. Accounting preserves prior effort and adds two regeneration units. Because evaluator success chooses the correction, this is an upper bound, not an online classifier.

Table 9: Held-out bounded corrective-retry audit. Parenthetical values give corrected initial failures.

Dataset	initial quality	final quality	corrected	mean added cost	correct broken
HotpotQA	.571	1.000	.429 (15/15)	2.31	0
QASPER	.700	.833	.133 (4/9)	2.71	0
All	.631	.923	.292 (19/24)	2.49	0

Seven Hotpot failures are repaired by increasing R , one by widening F , and seven require a compound fallback. One QASPER failure is repaired by increasing R ; eight invoke compound fallback, of which only three are solvable anywhere in the lattice. The audit establishes retry headroom and localizes missing capacity. It does not establish that an oracle-free diagnostic attains that rate. The endpoint remains direct initial prediction followed by at most one confidence-triggered, explicitly traced correction.

7.6 Candidate-normalized controller efficiency

Abstract controller cost is useful for comparing actions within this study, but it does not reveal how much of the available memory was searched or admitted. For each example, let N be the number of candidate source parents, E the number of evidence parents, K_{search} the unique parents visited by routing, and K_{admit} the unique parents whose native K/V enters the consumer. We report

$$R_E = \frac{|S_{admit} \cap G|}{E}, \quad P_E = \frac{|S_{admit} \cap G|}{K_{admit}}, \quad C_E = \mathbf{1}[G \subseteq S_{admit}], \quad (15)$$

together with admission fraction $\phi = K_{admit}/N$, working-set overhead $\omega = K_{admit}/E$, and the physical token count KV_{active} . Search comparisons, K_{search} , K_{admit} , and KV_{active} remain separate quantities.

Table 10: Held-out candidate-normalized efficiency. Learned policies are five-seed means; oracle and retry rows are evaluator-side one-seed ceilings. KV is mean admitted native-K/V tokens.

Dataset	Policy	R_E	P_E	C_E	K/N	K/E	KV
HotpotQA	R0 profile	.733	.400	.543	.697	1.905	979
	factorized oracle	.824	.624	.600	.576	1.633	840
	targeted retry	.948	.476	.886	.756	2.090	1079
QASPER	R0 profile	.750	.450	.700	.182	2.267	636
	factorized oracle	.917	.796	.833	.109	1.550	431
	targeted retry	.917	.588	.833	.190	2.517	675

The normalized view sharpens both positive and negative results. QASPER has a strong sparse factorized ceiling: its oracle improves recall and precision over R0 while admitting fewer candidates and 205 fewer native-K/V tokens on average. Targeted retry reaches that recall ceiling, but with more distractors and physical states. HotpotQA is less sparse at this parent granularity; retry buys substantially higher evidence recall by broadening admission, while the oracle identifies a smaller and more precise working set than R0. Across learned R1–R3A policies, lower K/N is accompanied by lower C_E : on HotpotQA complete recovery falls to .257–.291, and on QASPER to .620–.687. Their lower nominal cost therefore reflects under-allocation rather than a superior frontier.

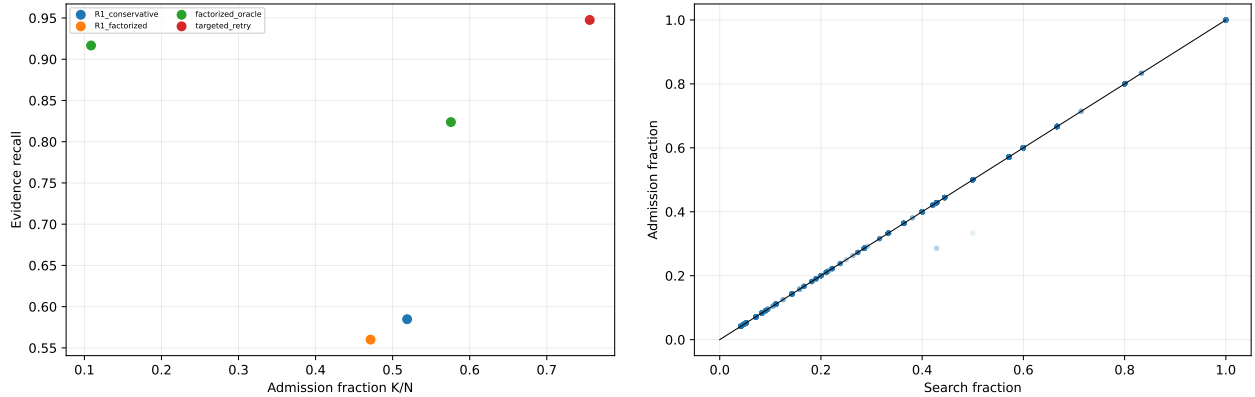


Figure 10: Left: evidence recall against admitted candidate fraction for the measured controller frontier. Right: row-level search and admission fractions; points below the diagonal search more parents than they materialize. Neither axis is an end-to-end latency measurement.

8 Retrieval Representation as an Adaptive Action

Effort control asks how broadly to search. Method control asks what evidence relation should define that search. We decompose one attempt into

$$\underbrace{(Q, F, S_r)}_{\text{interpret}} \longrightarrow \underbrace{(R, K, H, S_s)}_{\text{search}} \longrightarrow \underbrace{(B_s, B_{kv}, \theta, L, G, M)}_{\text{admit}}. \quad (16)$$

The root method S_r is one of semantic mean-gist, exact token, BM25, approximate token, or static hybrid search. The successor method S_s is native semantic, exact new-address, BM25 over the question/current state, approximate new-address, or state-conditioned hybrid search. Equation 16 permits, for example, a semantic root followed by an exact address, or a lexical root followed by native-semantic traversal. Searching two representations does not authorize twice as much K/V: B_s and B_{kv} remain separate decisions.

8.1 Protocol and action contract

The study imports Paper 2.6’s machine-readable action specification and verifies its SHA-256 digest before use. Its frozen corpus contains 58 validation and 74 held-out examples across QASPER, HotpotQA, 2Wiki, and MuSiQue. All 132 examples have matched two-chunk root rows; 120 have a labelled two-chunk successor decision, including 68 held-out examples. Twelve root-only examples remain in root and cross-dataset analyses but are not assigned a fictitious successor. One validation example omits ten of the 25 root/successor pairs; fixed-policy selection maximizes measured cohort coverage before quality, and missing actions are never imputed.

The serving contract is implemented in

`src/prahf/adaptive_search.py`

`AdaptiveSearchAction` begins at line 81 and carries independent methods around the existing factorized effort action. Strict action-spec import begins at line 188; dataset identity, gold geometry, oracle labels, and answer correctness are rejected from controller features at line 221. The deterministic method oracle at line 235 is evaluator-only. Useful-address scoring, targeted method-delta naming, and operation-specific accounting begin at lines 256, 286, and 314. The study runner derives oracle rows at line 221, evaluates policies at line 420, performs leave-one-dataset-out tests at line 509, and starts bounded method retry at line 639. These line references make the interpret/search/admit boundary independently reproducible without coupling it to the paper script.

8.2 Root and successor method headroom

Validation selects exact root search. Its held-out root recall is .066, while selecting the best measured root method independently per example reaches .104, a .038 absolute diagnostic gap. The 74 held-out root oracles choose exact for 21 examples, semantic for 19, BM25 for 15, approximate for 11, and hybrid for 8. No representation has majority support.

Conditioning successor choice on the per-example root oracle reaches .082 recall. Allowing the root/successor pair to vary jointly reaches .143, compared with .030 for the validation-selected fixed pair, BM25 root followed by native-semantic successor. The 68 successor oracles choose native semantic 22 times, exact new-address 15, BM25 state 14, approximate new-address 10, and hybrid state 7. Twenty-two of the 25 possible root-to-successor transitions occur at least once. The most common is semantic→native semantic, with only 9 examples. Representation choice is therefore state dependent, but the oracle is an upper bound, not a controller.

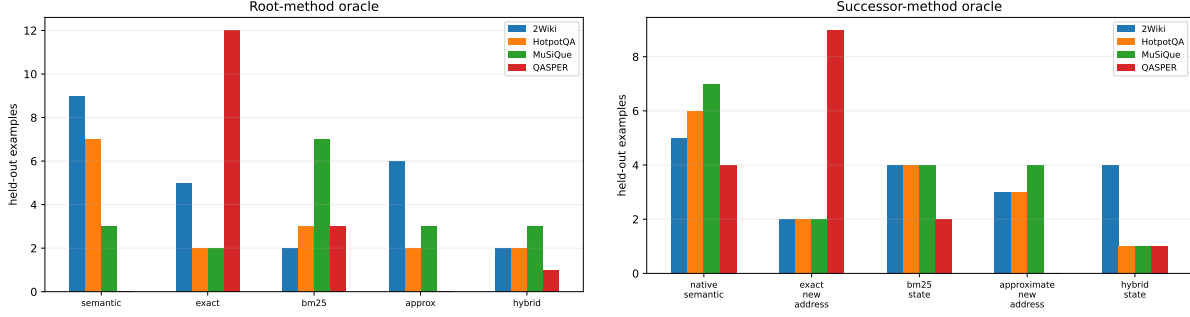


Figure 11: Held-out method-oracle distributions. Root (left) and successor (right) choices are heterogeneous across all four datasets; counts are identities, not repeated model seeds.

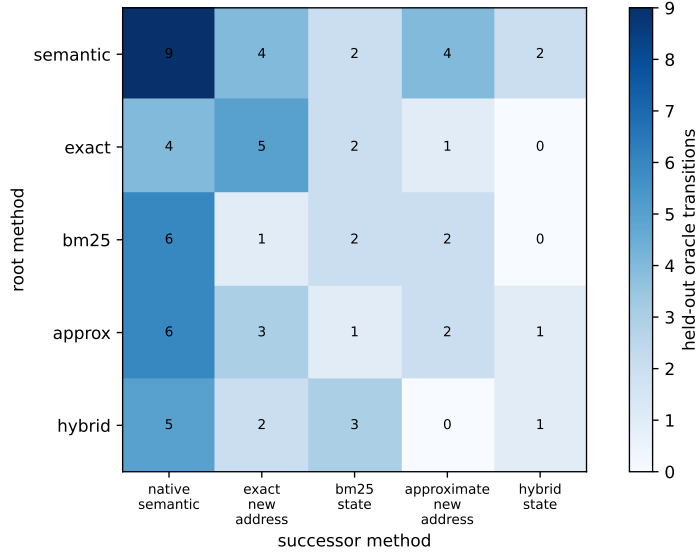


Figure 12: Held-out joint-oracle transitions. Sparse mass across 22 cells argues against tying root and successor representations, but does not say that an observable selector can predict the cell.

8.3 Fixed, cascaded, and learned policies

Table 11 evaluates downstream evidence discovery rather than method-label accuracy. S0 forces the corresponding root and successor representation; S1 selects one pair per dataset on validation and is diagnostic because dataset identity is forbidden at deployment; S2 is static hybrid; S3 is one global root/successor pair; S4 and S5 adapt one stage; S6 predicts both; and S7 is the per-example oracle. C0 is a non-learned cascade over exact/BM25 gaps and disagreement.

Table 11: Held-out matched-budget method policies over 68 successor-bearing examples. Comparisons are evidence recall, precision, and MRR. No policy completes every broad evidence span within the four requested chunks, so complete-recovery is zero and omitted.

ID	policy	recall	precision	MRR
S0	one method at both stages	.011	.044	.179
S1	dataset-fixed diagnostic	.028	.081	.234
S2	static hybrid	.035	.088	.229
S3	fixed BM25→native pair	.030	.081	.229
S4	adaptive root only	.037	.081	.212
S5	adaptive successor only	.024	.074	.208
S6	adaptive root and successor	.032	.074	.225
C0	interpretable cascade	.048	.118	.240
S7	per-example method oracle	.143	.353	.615

The observable learned controller does not close the gap. Adaptive root selection adds .007 recall over S3, but successor-only selection loses .006 and joint selection adds only .002. The cascade is the strongest deployable row, yet its gains are concentrated in HotpotQA (.089) and 2Wiki (.078); it falls below S3 on QASPER and MuSiQue. Static hybrid also remains below the cascade. The result is not “adaptivity works” in general. It is the narrower claim that method headroom exists and these small selectors are not yet reliable enough to exploit it.

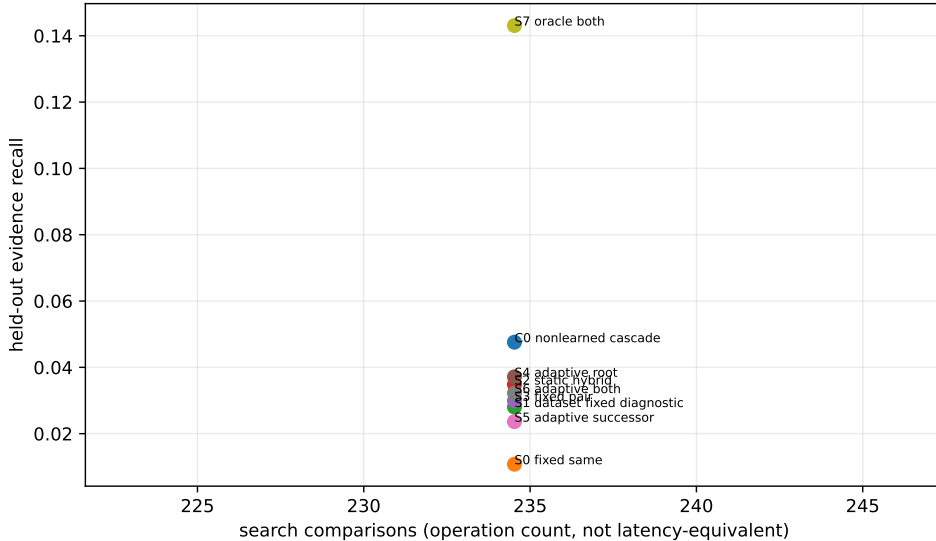


Figure 13: Held-out method policies. The horizontal axis shows search-operation counts and is not a latency-equivalent scalar across semantic GEMM, postings, and token-span work.

8.4 Generalization, self-query state, and useful addresses

Leaving each dataset out in turn gives root-method classification accuracy .167 on 2Wiki, .188 on HotpotQA, .278 on MuSiQue, and .250 on QASPER. Dataset identity is absent from every feature vector. These values are too weak to support a general channel controller. The existing frozen `S2_embed_last` cache covers only 32 HotpotQA/QASPER identities, all assigned to the Paper-2.6 test split. We therefore report four-fold identity-grouped out-of-fold classification rather than

mislabeling a fitted-on-test result as held out. Observable features reach .313 root-method accuracy, the embedding reaches .438, and their concatenation returns to .313. This is a small classification signal, not a measured downstream-quality gain or a reason to replace R0.

Only 12.2% of held-out examples satisfy the evaluator-side useful-address predicate. A validation-trained observable gate predicts no positive at threshold .5, hence both precision and recall are zero. Abstention avoids confident lexical expansion but misses all useful cases. The gate must improve before exact or approximate successor following can be enabled by default.

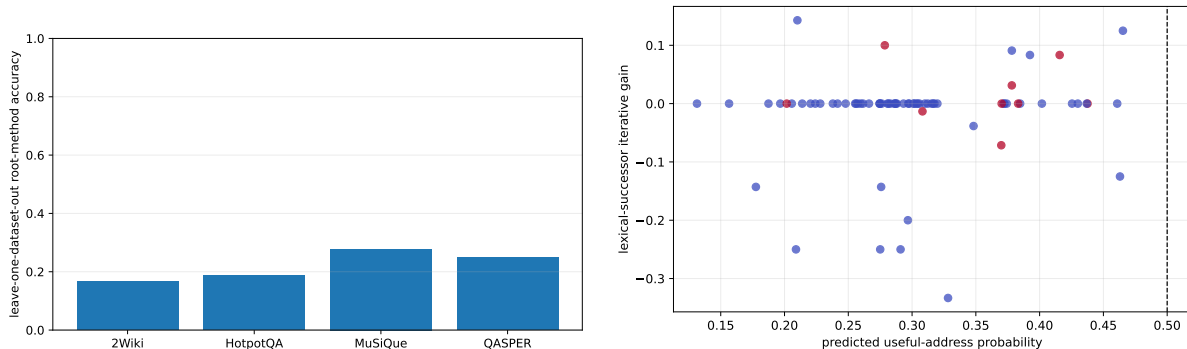


Figure 14: Left: leave-one-dataset-out method accuracy without dataset identity. Right: predicted useful-address probability versus lexical-successor gain; the fitted gate abstains at .5.

Query-region interaction is not identifiable in this inherited corpus: 130 examples use layout code 0 and only two use code 1. We therefore do not add the interaction-aware architecture proposed in the study plan. The controlled query-region fixtures remain evidence that interpretation matters, but they cannot identify a natural query-region $\times$ method effect.

8.5 Method retry and the factorized surface

From S6, 32 held-out examples need no oracle correction, 16 need only a successor-method switch, 8 need only a root-method switch, and 12 need both. Always switching reduces recall from .032 to .023. A validation-trained delta controller predicts both action type and replacement methods; it reaches .037 recall with .221 action accuracy. Oracle correction reaches .143. Method switching is thus a valid retry action, but evaluator-side correction remains much stronger than the learned policy.

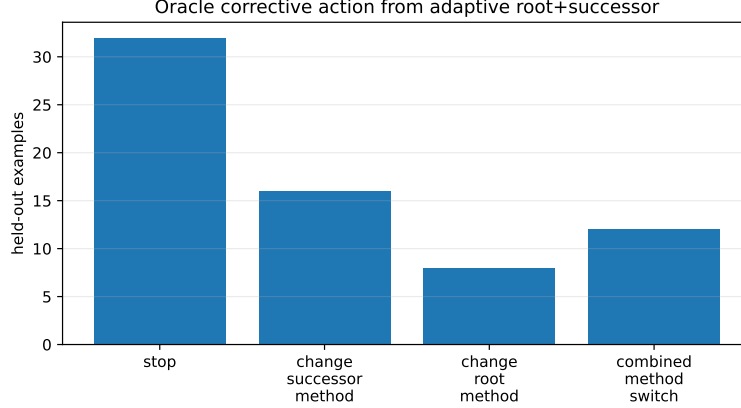


Figure 15: Evaluator-side correction decomposition from adaptive root+successor selection. The learned retry predicts the delta and replacement methods separately; it never receives gold evidence.

All 32 Paper-3.5 factorized identities overlap the channel corpus, although their validation/test assignment differs. We join them by stable dataset/example identity and retain both measured surfaces: factorized chain quality and cost, and Paper-2.6 method recall and operation counts. The reported joint floor is their minimum. It is a composition diagnostic, not execution of every method inside all 792 factorized actions. Accordingly, factorization cost savings and adaptive method quality gains are plotted on separate axes and are never added. Paper 2.6 materializes no K/V; the inherited Paper-3.5 rows alone supply physical K/V counts.

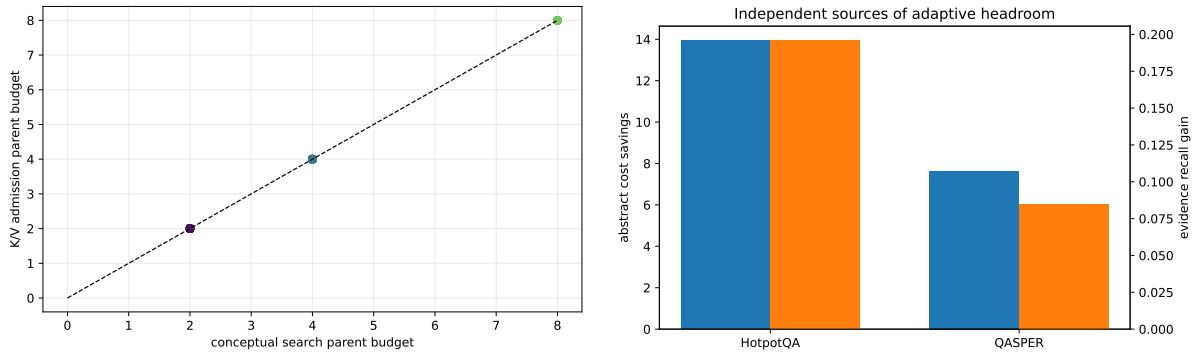


Figure 16: Left: conceptual parent budget versus physical admission budget on the identity-matched factorized surface. Right: factorized cost savings and method recall gain remain distinct units.

Semantic comparisons, BM25 postings, exact/approximate span operations, index memory, and measured prototype latency are retained in separate columns. The exhaustive Python timings remain unsuitable for serving claims. A production controller should optimize measured downstream quality under per-system resource constraints, not a fabricated scalar that treats one posting lookup as one GPU dot product.

9 Strengthened Request/Reply and a Single Root-State Callback

The preceding controller selected how many cached facets to use, but not how those facets were constructed. That omission would make a callback comparison unfair: a callback with semantic graphs would have a larger interpretation space than its one-shot baseline. We therefore first add the graph action to request/reply, then execute the same root/successor surface with and without one post-root callback.

9.1 Five facet modes and one hierarchical representation

The one-shot action now chooses

$$F_m \in \{\text{global}, \text{syntactic}, \text{multiscale}, \text{graph}, \text{syntactic} \rightarrow \text{graph}\}. \quad (17)$$

Global uses the final contextual query state. Syntactic pools deterministic clauses and relation neighborhoods. Multiscale pools overlapping windows. Graph constructs a sparse top- k graph over contextual query tokens and pools connected components. The hierarchical mode first partitions sentences, clauses, delimiters, or fixed regions and then constructs a local semantic graph inside each region. It retains the global root, coarse regions, and graph children as distinct scoring views.

Each tree node records its parent and children, possibly non-contiguous token indices and spans, embedding and centroid, entities, query-rare tokens, lexical statistics, component density and weight, and confidence. This is provenance, not hidden evaluator state. The graph sees one already contextualized query pass; it neither re-encodes windows nor inspects evidence labels. Component minimum and maximum sizes are enforced after clustering and before pooling.

The reusable Paper-2.7 graph primitives are in `query_graph.py`, `query_graph_cluster.py`, and `query_graph_facets.py`. The adaptive contract begins with `GraphFacetConfig` at `adaptive_facets.py:72` and the auditable tree node at line 111. Graph-region construction begins at line 323; the five-mode builder begins at line 400. `AdaptiveSearchAction` at `adaptive_search.py:81` adds facet mode, coarse partition, graph similarity, threshold, top- k , component bounds, fusion, channel profile, and per-facet policy to the existing independent root/successor action. Existing callers remain global/fixed by default.

9.2 Per-facet search methods

Heterogeneous facets need not share one retrieval representation. A route now has the form

$$(q_f, S_{r,f}, S_{s,f}, K_{r,f}, K_{s,f}). \quad (18)$$

The fixed baseline applies one pair to every facet. The type rule sends entity/rare-identifier facets to exact or BM25 search, relational facets to semantic search, and mixed facets to hybrid search. A ridge classifier predicts a root/successor pair from lexical, structural, confidence, and graph metadata. The oracle chooses the best measured pair independently per facet and is never available to serving code. `FacetRoute` is defined at `facet_routing.py:31`, the linear selector at line 79, the evaluator-only oracle at line 139, and common routing dispatch at line 168.

9.3 Frozen replay and graph accounting

We rerun the 74 identity-frozen Paper-2.7 test queries on CUDA: 16 QASPER, 16 HotpotQA, 24 2Wiki, and 18 MuSiQue. Qwen3-0.6B query states, memory gists, chunk identities, and positives are reused; no model weights are updated and no K/V is materialized. Every condition requests four

32-token chunks. The selected graph policy is contextual similarity, union symmetrization, top-2, threshold .45, and connected components, selected previously on controlled validation. Paper-2.6 global and hybrid selections reproduce exactly.

Table 12 reports semantic-only graph rows. Hierarchical faceting preserves or nearly preserves global-graph recall. Its main measurable advantage appears on MuSiQue, where partitioning reduces pairwise similarity work from 796 to 570; it makes 1.33 local graph calls and introduces .384 mean parent/child facet overlap. Hard global graph communities are disjoint, hence their overlap is zero. Python/CUDA construction times include synchronization and vary across runs; they characterize this prototype, not a production speedup.

Table 12: Graph versus hierarchical syntactic-to-graph root discovery. Work is mean pairwise similarity evaluations; time is graph construction plus clustering in ms/example.

Dataset	graph recall	hier. recall	graph work	hier. work	graph ms	hier. ms
QASPER	.033	.064	90	90	31.9	18.8
HotpotQA	.234	.212	524	524	70.2	35.5
2Wiki	.175	.175	288	288	40.1	31.1
MuSiQue	.090	.092	796	570	39.1	36.1

Adding graph facets to the inherited hybrid channel changes recall by $-.0078$ on QASPER, $+.0156$ on HotpotQA, 0 on 2Wiki, and $-.0078$ on MuSiQue. Hierarchical hybrid has the same recall as global-graph hybrid on these four aggregates. Thus graph construction adds useful example-level alternatives but is not an unconditional replacement for the global hybrid channel.

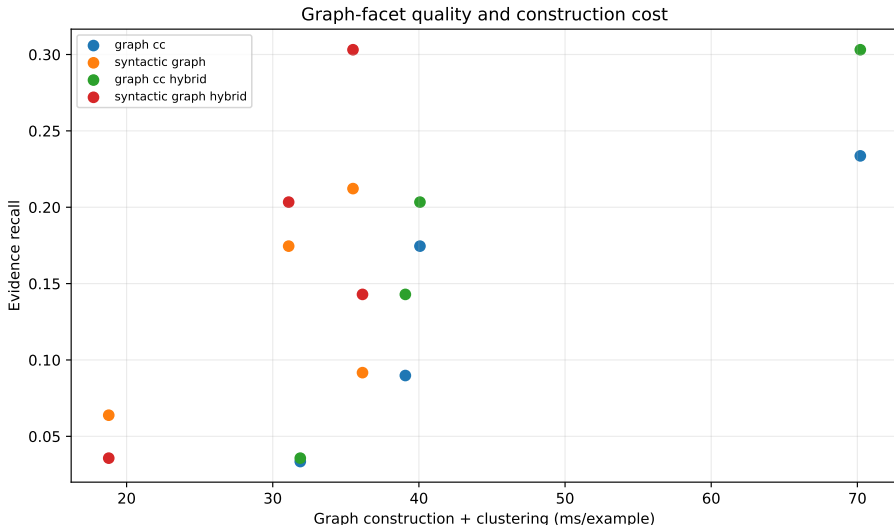


Figure 17: Measured graph-facet quality and construction cost by dataset. Points are aggregate root-discovery conditions, not generated-answer quality or optimized serving latency.

9.4 Five-seed one-shot and per-facet controllers

The Paper-2.7 cohort was originally a test cohort. To test learnability without reusing the same identities for fitting and evaluation, we create an explicitly post-hoc, dataset-stratified 45/29 identity split. This is an exploratory controller split, not a new untouched benchmark test set. Fourteen observable query/facet aggregates are used; dataset identity and evidence labels are absent. Five

bootstrap seeds train each ridge selector. B2 chooses facet construction under semantic root search. B3 chooses facet construction and root method, with the matched successor name carried in the action contract. B4 chooses methods per facet and fuses four chunks by reciprocal rank. B7 is the best measured whole-query one-shot action.

Table 13: Held-out root discovery at four chunks. Learned rows average five seeds; B7 is one evaluator-only oracle. B3 does not claim a newly executed full successor traversal.

ID	policy	recall	precision	MRR
B2	learned graph-facet choice	.117	.222	.323
B3	learned facet/root action	.179	.303	.548
B4	learned per-facet methods	.198	.328	.621
B7	whole-query one-shot oracle	.273	.457	.704

B2 underperforms the existing global-channel request/reply controller (.183 recall): choosing facet construction alone is too difficult for this small feature set. Opening the root channel restores most of that loss. Per-facet learning improves mean recall by .018 over B3, with standard deviation .020 and a .179–.231 seed range. The remaining B7–B3 gap is .093 recall. The action space is therefore useful, but one-shot action prediction remains the limiting component.

The per-facet controls sharpen that conclusion. Fixed semantic fusion reaches .120 recall, the hand-written type rule .224, learned routing $.198 \pm .020$, and the evaluator-only per-facet oracle .283. The simple type rule exceeds the learned mean on this small split, while the oracle shows that channel heterogeneity exists below the whole-query level. This is evidence for better calibration and training data, not for making the oracle or dataset identity a serving feature.

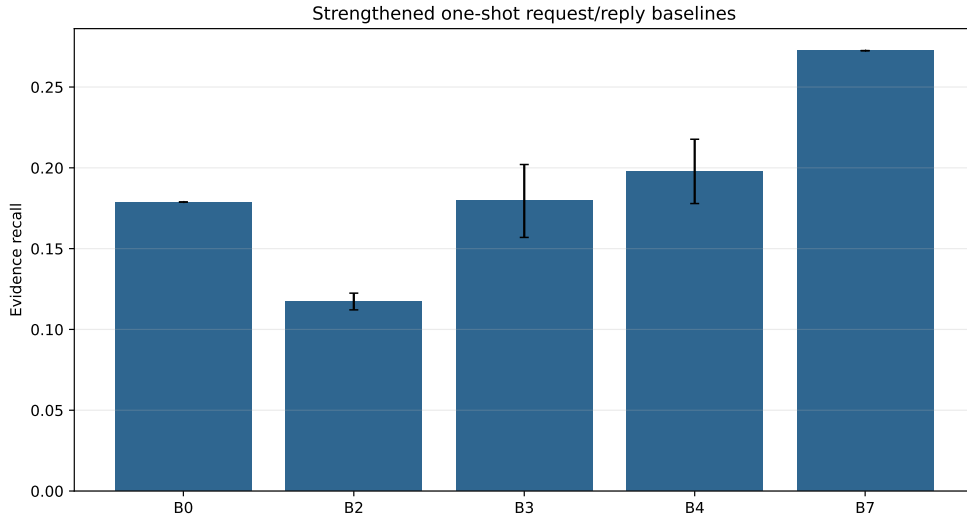


Figure 18: Strengthened request/reply baselines. Error bars are across five router seeds where applicable; static and oracle rows have one deterministic value.

The inherited full root/successor study remains a different measurement scope: S6 recall is .032 and its oracle is .143 over 68 successor-bearing examples. We preserve those rows separately and do not compare their magnitudes directly with the preceding four-chunk root-discovery replay. The next experiment closes that scope gap by executing every facet/root/successor action at one fixed two-root plus two-successor budget.

9.5 One bounded ROOT_SELECTED event

The serving contract permits exactly one event:

$$q \rightarrow a_0 \rightarrow \mathcal{R}_1 \rightarrow z_{\text{root}} \xrightarrow{\text{ROOT_SELECTED}} d_1 \rightarrow \mathcal{S}_1. \quad (19)$$

Here a_0 chooses facet and root method, $\mathcal{R}_1$ is the selected root set, z_{root} contains observable post-search state, and d_1 chooses one successor method or conditional graph refinement. There is no second event and no generation retry. The callback cannot inspect dataset identity, gold evidence, or answers.

The state records ordered root identities and scores, top-score gap, candidate entropy, channel agreement, newly exposed lexical addresses, facet agreement, root dispersion, mean root gist, and remaining conceptual and K/V budgets. `RootState` begins at `src/prahf/root_callback.py:48`, the bounded `RootDecision` at line 145, and the single-event dispatcher at line 207. The dispatcher calls a root executor once, invokes `on_root_selected` once, and passes the decision to a successor executor. Existing static execution remains unchanged unless a controller is supplied.

9.6 Complete measured action surface

We retain the original 58 validation and 74 test identities rather than creating another post-hoc split. Each identity executes nine facet profiles: global with one facet, and structural, multiscale, graph, and structural-to-graph with two and four requested facets. Crossing these with five root methods and five successor methods gives

$$132 \times 9 \times 5 \times 5 = 29,700 \quad (20)$$

measured actions. Every action retains two roots, follows two successors, and admits at most four 32-token conceptual chunks. Thus quality comparisons do not hide broader callback search or K/V admission. Root search persists 5,940 reusable states; fitting a new callback does not repeat retrieval. Root-state construction is at `experiments/paper3_5_adaptive_pra/request_reply_full_surface.py:215` and successor execution at line 280. The latter caches duplicate root/channel score computations across actions but emits the same rankings and per-action cost estimates; a checked smoke replay reproduced every selection and quality field exactly.

Graph construction is not free. On the test partition, global, structural, multiscale, graph, and structural-to-graph construction take .43, 1.86, 18.85, 32.22, and 33.90 prototype ms/example. The global graph performs 424.5 pairwise evaluations in one call. Structural-to-graph reduces this to 368.0 evaluations across 1.08 local calls, but control overhead removes a wall-clock advantage in the Python prototype. Its mean parent/child overlap is .564. These timings identify work; they do not establish optimized serving latency.

9.7 Matched one-shot and callback results

Table 14 is the controlled 2×2 comparison. B1 and B3 exclude graph; B2 and B4 permit it initially. B5 permits conditional graph refinement. B5t is a transparent validation-tuned uncertainty rule over root gap, entropy, and channel disagreement. Learned rows average five bootstrap seeds; dataset and evaluator fields are absent. All rows have the same .0719 active conceptual fraction and 225.3 mean prototype comparisons.

Table 14: Full-surface test retrieval. Values are evidence recall; learned and threshold rows show mean $\pm$ standard deviation across five seeds. B6/B7 are evaluator-only ceilings.

ID	policy	recall
B0	validation-fixed global request/reply	.205
B1	query-only one-shot, no graph	.180 $\pm$.022
B2	query-only one-shot, graph available	.180 $\pm$.023
B3	root callback, no graph	.168 $\pm$.021
B4	root callback, graph available initially	.167 $\pm$.019
B5	learned conditional graph refinement	.167 $\pm$.021
B5t	threshold conditional graph refinement	.188 $\pm$.024
B6	complete-action oracle	.365
B7	stage-wise oracle	.365

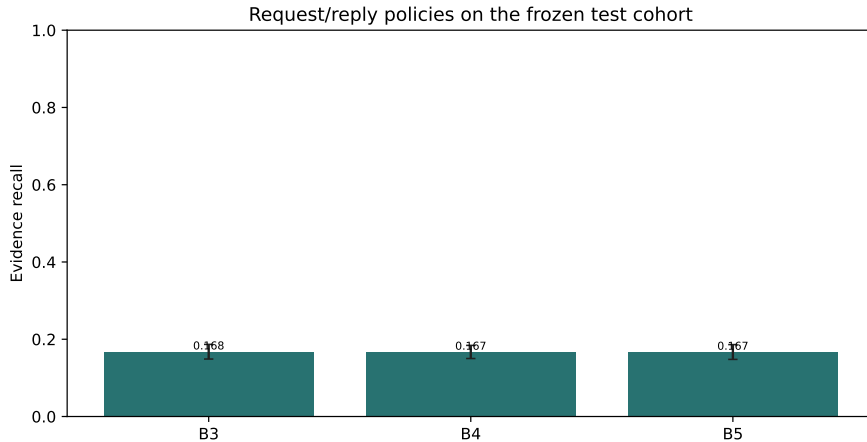


Figure 19: Matched request/reply results on all 74 frozen test identities. Error bars show variation across controller seeds, not example-level uncertainty.

The result does not support the callback hypothesis. B3–B5 trail their matched one-shot controls. Adding the compact root gist to callback features raises B5 to .177 in an ablation, still below B1. The threshold controller is stronger: it gains .008 recall over B1, correcting 21.6% and harming 17.6% of test decisions while graph-refining 16.5%. It nevertheless remains below B0. Three of five threshold seeds invoke no graph refinement; the gain is partly a simpler factorization of initial and successor choices, not a stable value-of-information result for graphs.

The oracle result is more fundamental. B6 chooses the best complete measured action per example. B7 first chooses the root action and then its best successor from the same Cartesian surface. They must coincide when both evaluator oracles optimize the same realized rows, and they do: both reach .3649 recall, .5777 precision, .7964 MRR, and .0270 complete recovery. Calling their difference an intrinsic sequential gain would be incorrect. The oracle chooses global facets for 83.8% of examples, structural facets for 13.5%, and graph facets for 2.7%; no other graph profile survives the deterministic quality/cost tie-break.

Leave-one-dataset-out transfer strengthens the negative conclusion. B1, B2, B3, and B5t reach .167, .170, .155, and .180 recall, respectively, versus .190 for the validation-fixed policy and .365 for the oracle. QASPER is the weakest transfer case: the same rows reach .029, .031, .035, and .042, compared with a .200 oracle. More controller capacity is not warranted before features or

supervision change.

9.8 Native-K/V and frozen decode confirmation

A retrieval score does not prove that chosen identities reach the consumer. We therefore select two deterministic test examples per dataset and compare no memory, B0, B1, B3, B5t, and B6 with frozen Qwen3-0.6B. The live adapter maps each frozen conceptual span explicitly to its native cache span; it never assumes that experimental and runtime chunk identifiers match. The mapping begins at `experiments/paper3_5_adaptive_pra/request_reply_end_to_end.py:124` and live execution at line 208. All memory conditions map four conceptual chunks to four physical chunks, materialize 128 unique native-K/V tokens across layers 20–27, expose 1,024 token-layer states (4 MiB), and activate 7.75% of source tokens on average.

Table 15: Eight-example native-K/V decode confirmation. $\Delta \log p$ is mean gold-token log probability relative to no memory. The sample is a mechanism check, not a benchmark estimate.

Condition	retrieval recall	exact/F1	$\Delta \log p$	K/V tokens
No memory	.000	.375	.000	0
B0 fixed	.279	.375	+.567	128
B1 one-shot	.358	.500	+.770	128
B3 callback	.228	.375	+.912	128
B5t threshold	.360	.500	+.687	128
B6 oracle	.533	.500	+.639	128

Every memory condition improves gold-token likelihood, confirming that selected native K/V enters the frozen consumer. Generated exact/F1 improves for B1 and B5t but does not increase monotonically with retrieval recall; B6 ties them, and B3 obtains the largest likelihood gain without improving generated accuracy. With eight examples, that non-monotonicity is a warning against substituting retrieval metrics for answer quality. The SDK therefore keeps frozen request/reply as the default and exposes the callback as research-only, opt-in control.

10 Self-Encoded Query Routing: Letting the Backbone Read the Task Before Allocating Memory

Instead of immediately adding a separate semantic encoder, the backbone can briefly represent its own task:

$$Q \longrightarrow z_Q^{(k)} \longrightarrow \hat{a} \longrightarrow \text{PRA search/admission/consumption.} \quad (21)$$

This duplicates only a short question prefix in D0 and can avoid duplication entirely in D1 when $z_Q^{(k)}$ is an exact prefix of normal execution. The empirical question is whether that state predicts sufficient effort better than cheap observable geometry.

10.1 Representation and layer gate

Table 16 reports the profile-target gate. Validation CV selects the last token embedding, available before any decoder block. Its held-out quality is .668, however, below the observable router’s .702; signed hashing is lower at .609. The validation-selected contextual variant is layer-14 hidden mean and reaches only .674 while paying a 256-token prefix. Consequently the experiment stops before a compact external encoder.

Table 16: Five-seed profile routing by representation. CV quality selects representations; held-out quality is never used for selection. Cost includes Eq. 11; latency is the cumulative query representation time on the measured GPU.

Representation	CV quality	test quality	total cost	under	latency (ms)
S0 observables	.728	.702	28.25	.222	0.0
S1 signed hash	.686	.609	25.53	.314	0.0
S2 embedding last	.726	.668	27.20	.255	.015
S2 embedding mean	.699	.717	25.74	.206	.015
S3 hidden layer 7 mean	.714	.698	27.13	.225	30.8
S7 native K layer 13 mean	.718	.655	27.85	.268	59.3
S8 contextual hidden layer 14 mean	.705	.674	27.70	.249	177.5

S2 embedding mean is numerically highest on held-out units (.717), but its CV quality is .699 and it was not selected. Treating the held-out maximum as the winner would leak the test split. The appropriate conclusion is therefore not that embeddings improve R0, but that a larger grouped cohort is needed to resolve a small, unstable difference. The reversal between validation-selected embedding-last and held-out embedding-mean also makes pooling instability explicit; no learned pooling sweep is warranted on this cohort.

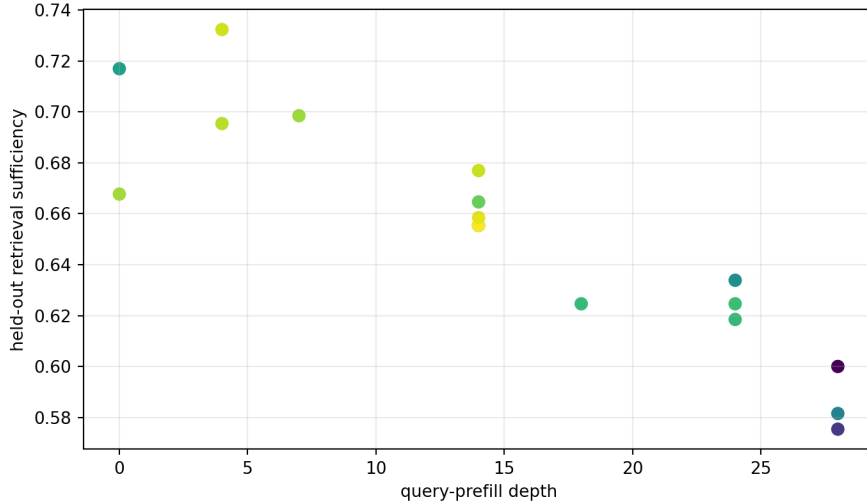


Figure 20: Held-out profile-router quality over query-prefill depth. Color encodes total abstract cost. Greater depth does not produce a monotonic quality gain.

The validation-selected embedding state separates the datasets imperfectly. Query plus static observables improves HotpotQA over query-only routing (.646 versus .571), but slightly reduces QASPER (.693 versus .733). Runtime fields such as root gaps after search remain retry-only and are not admitted to the initial router. The current sample does not support a universal fusion rule.

10.2 Query-only versus contextual geometry

Context changes pooled query geometry less than its 256-token cost might suggest. Mean cosine similarities between query-only and contextual states are .945 at hidden layer 14, .956 at the final hidden state, .921 for native Q at layer 13, and .966 for native Q at layer 23. The small drift does

not translate into better validation-selected control. This supports query-only representation as the cheaper diagnostic substrate, but not the stronger claim that the current head can use it.

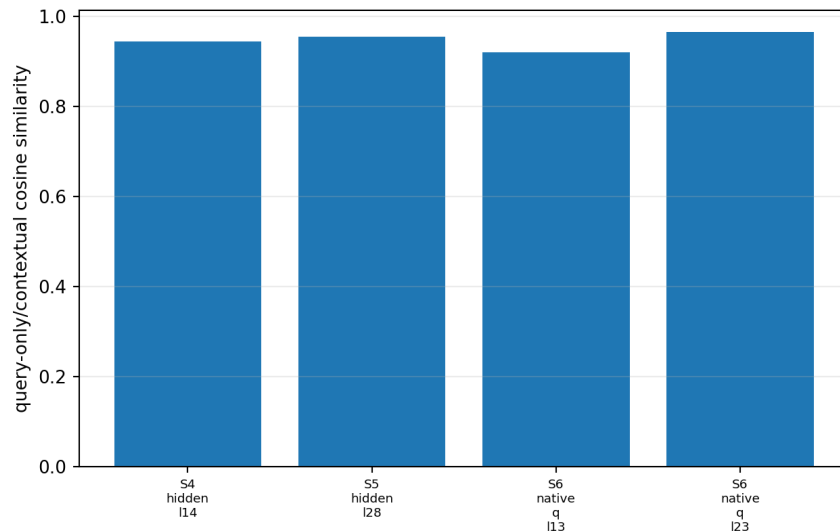


Figure 21: Cosine similarity between pooled query-only and 256-token source-contextualized question states. Contextual processing is expensive, while the selected representations remain close.

10.3 Coherent modes versus parameter values

We next hold the validation-selected embedding-last representation fixed and replace targets. Four validation-derived global profiles and five coherent search profiles are fitted without held-out labels. Table 17 shows that all four target families under-allocate. The independent and interaction-aware heads tie at .523, below frozen-profile routing with the same representation (.668). Group profiles are cheapest only because they miss 45.8% of oracle-solvable units. Negative nominal regret in these rows is therefore failure, not efficiency.

Table 17: Target granularity with validation-selected S2 embedding-last. Global and group code-books are derived on validation only.

Target architecture	quality	total cost	under	over
Global profiles	.517	14.29	.406	.277
Interpret/search/admit groups	.465	12.73	.458	.271
Independent parameter values	.523	15.83	.400	.366
Autoregressive interactions	.523	16.12	.400	.363

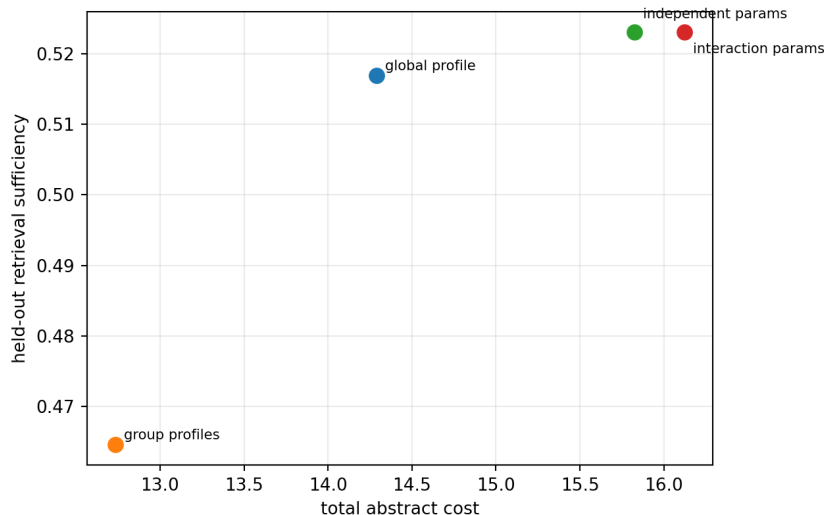


Figure 22: Target-architecture quality/cost frontier. Parameter heads improve over grouped modes, but none approaches the robust profile selector; low-cost group routing is under-allocation.

10.4 Double encoding and exact reuse

The selected embedding state appears before every decoder block and therefore before the earliest layer-24 PRA consumer. A fresh D0 prepass costs .020 abstract units, processes 35.2 query-prompt tokens on average, and takes .015 ms on the measured GPU. D1 reuses the same normal embedding state; exact prefix-continuation tests remove recomputation and reduce total cost from 27.20 to 27.18 without changing the .668 routing endpoint. This reuse result is architectural, not a quality improvement.

Reuse is invalid after a memory-consuming layer, when ordinary context should precede the query, or after query reinterpretation changes the selected span. The 216 controlled wrong-to-correct reinterpretations therefore require one new encoding of the corrected cue; repeatedly increasing F , R , K , H , or B around a known-wrong cue is not an equivalent substitute.

11 Inherited Runtime Measurements and Scope Handoff

The following measurements were completed during the original combined control/serving study and are retained for provenance. They are not Paper 3.5 contributions. Kernel compilation, memory hierarchy, cache/prefetch policy, batching, concurrency, and broad engine comparison now belong to Paper 5.5; first-class vLLM logical blocks and scheduling belong to Paper 6.

11.1 Exact GEMM is the current small-index default

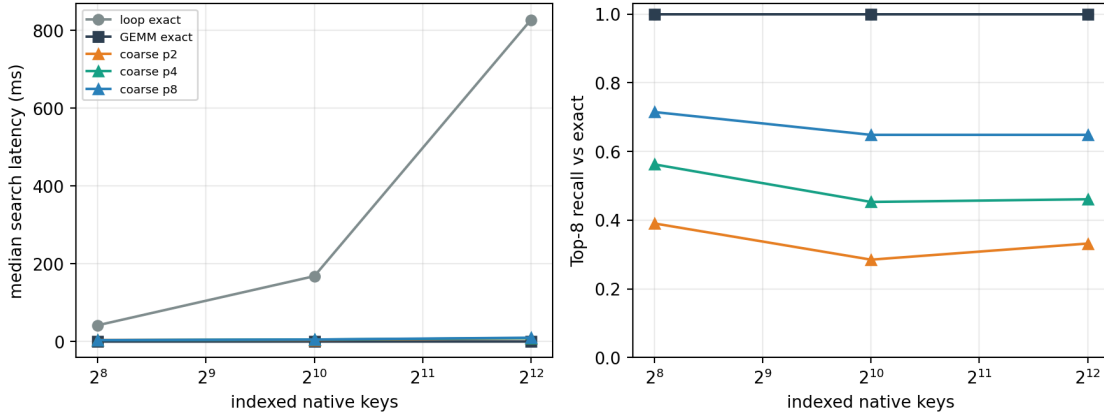


Figure 23: Native-Q/K search. Exact GEMM removes Python per-key overhead. The present deterministic coarse index loses recall and does not yet beat GEMM at these memory sizes.

At 4,096 keys, the Python loop takes .827 seconds for 32 queries while exact GEMM takes .327 ms, a 2,527 $\times$ speedup with identical Top-8 results. Coarse-to-fine search with two, four, and eight probes reaches .332, .461, and .648 recall relative to exact, respectively, while taking 8–13 ms. The ANN path is therefore not adopted as a default. It requires a mature FAISS/HNSW/GPU index and larger-scale crossover measurements before it can support a speed claim.

11.2 Gather and page layout

Table 18: Median standalone CPU gather time in milliseconds. Every row has exact K/V parity.

Source tokens	Python slice/cat	fused index-select
512	.197	.182
2,048	.195	.227
8,192	.191	.198

The one-index path has exact parity but no consistent CPU speed advantage in this small benchmark; it is not a fused CUDA kernel. The fixed-page Python lookup is substantially slower: depending on page size and selection dispersion, it costs 14–37 $\times$ contiguous `index.select`. Pages of 16–128 tokens waste 15–127 tail slots for the 1,537-token request. Clustered selection touches 9, 5, 3, and 2 pages; dispersed selection touches 97, 49, 25, and 13. These numbers explain what a production page gather should optimize, not how fast that gather will be.

11.3 Cache and prefetch

Without prefetch, the best controlled hit rate is .921 at 32 pages; LRU, LFU, and hybrid tie because the full working set nearly fits. At 16 pages LFU reaches .896, above LRU’s .833, because the workload contains a stable hot set plus a moving path. Graph-neighbor prefetch is measured, while the next-access trace is reported only as an oracle upper bound and never as a deployable predictor. Useful and wasted prefetch transfers are explicit artifact columns.

11.4 Heterogeneous batching

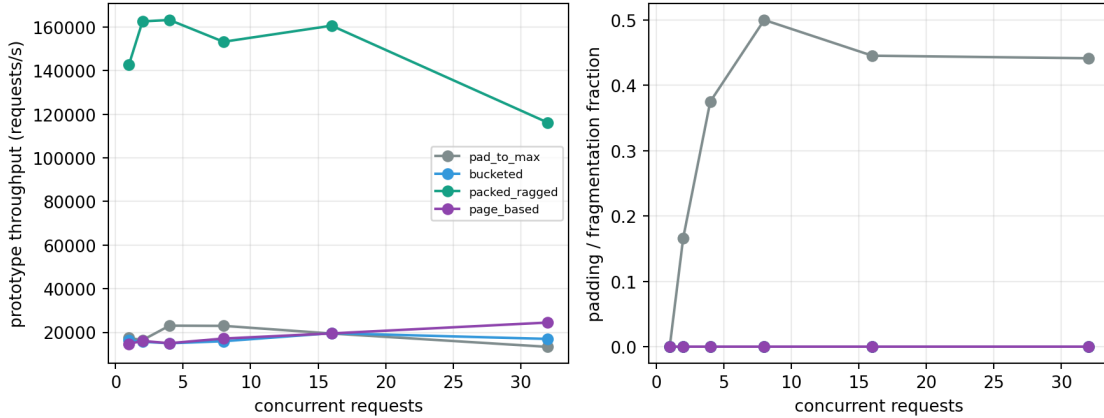


Figure 24: Controlled CPU batching. Packed ragged execution avoids padding and is fastest for this small per-request matrix-vector workload; this is not a GPU attention-kernel benchmark.

At concurrency 32, pad-to-maximum wastes 44.1% of allocated token slots. Bucketed, page-based, and packed layouts have zero waste for the chosen lengths. Packed per-request operations reach 128,154 requests/s versus 13,273 for pad-to-maximum in this CPU microbenchmark. On GPU, launch overhead and fused kernels can reverse this ordering; the artifact therefore records device and measurement scope.

11.5 Inherited Qwen serving observations

Table 19: Mean held-out single-request Qwen observations inherited from Gate 3. Active K/V counts layer-token states, not unique source tokens.

Dataset	Effort	active K/V states	TTFT (s)	total (s)
2Wiki	E0	2,200	.169	1.000
2Wiki	E1	4,267	.190	1.348
2Wiki	E2	6,215	.198	1.465
MuSiQue	E0	7,562	.320	3.740
MuSiQue	E1	44,683	.705	7.688
MuSiQue	E2	47,869	.737	7.936

These rows demonstrate that effort can materially change K/V and latency. They do not demonstrate that the new controller produces the same values end to end, because the adaptive controller was evaluated on frozen routing traces rather than a live Qwen scheduler. GPU utilization and dollar cost were not captured on the local hardware and remain explicitly missing.

12 Matched Baselines

12.1 RAG, iterative retrieval, and long context

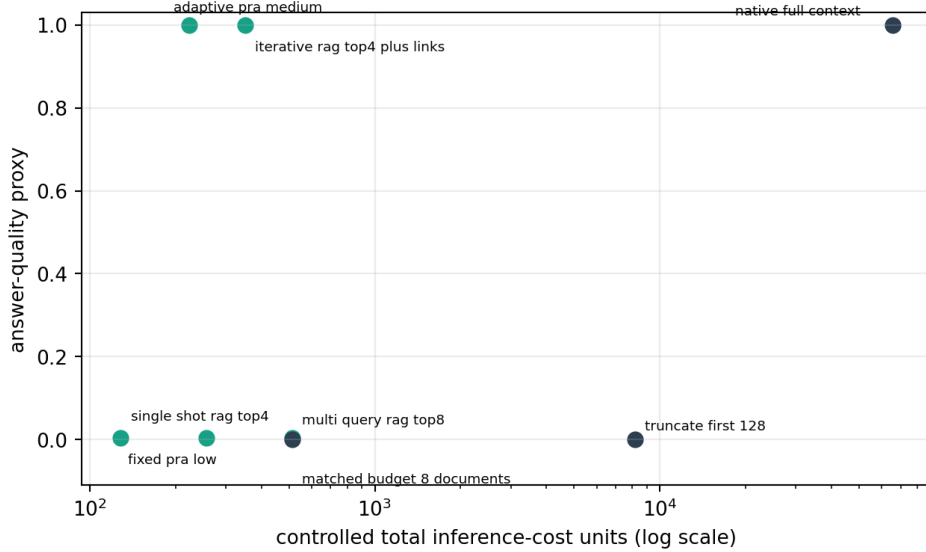


Figure 25: Controlled two-hop quality/cost frontier. Iterative RAG is the appropriate comparison for iterative PRA; a one-shot RAG baseline is insufficient.

Table 20: Controlled matched-corpus retrieval results. Quality is target-evidence recovery.

Method	quality	active K/V	cost units
one-shot RAG Top-4	.005	256	256
multi-query RAG Top-8	.005	512	512
iterative RAG Top-4 + links	1.000	320	352
fixed PRA low	.005	128	128
adaptive PRA medium	1.000	192	224
native full context	1.000	65,536	65,536

The corpus is constructed so the first retrieval identifies a bridge and the answer lies behind its link. One-shot methods fail by design; iterative RAG and adaptive PRA recover the target. PRA uses fewer controlled active tokens because it activates native memory rather than serializing all selected documents into a prompt. This supports the accounting distinction, not a general claim that PRA dominates optimized iterative RAG.

12.2 K/V selection proxies

At budget 32, the H2O, SnapKV, StreamingLLM, ClusterKV, and native-Q/K proxies recover the planted salient token with rates .915, 1.000, .120, .045, and 1.000. At budget 128 the rates are 1.000, 1.000, .240, .255, and 1.000. These outcomes reflect the controlled score construction. They test matched-budget bookkeeping and expose the expected failure of recent-only selection when evidence is not recent. They do not rank the upstream systems on real language generation.

Dense and external retrieval solve different physical problems. H2O and SnapKV select or compress an existing sequence cache; RAG retrieves text for prompt ingestion; PRA selects addressable contextual K/V. Longformer [2], BigBird [16], Reformer [4], and Routing Transformer [12] change the attention architecture and require compatible checkpoints. Compressive [11], memorizing [14], Infini-Attention [10], Landmark Attention [9], RETRO [3], and Titans [1] use recurrent, compressed, or external memory. We preserve this taxonomy rather than forcing unfair experimental rows.

13 SDK and Reimplementation Contract

The runtime is exposed through Python data classes and the `pra-hf adaptive` CLI. A caller can inspect profiles or resolve an automatic plan:

```
pra-hf adaptive profiles --output adaptive_effort_profiles.json
pra-hf adaptive plan --mode auto --features request_features.json \
  --controller controller.json --max-retries 2 \
  --max-search-budget 8 --max-active-kv 1024 \
  --latency-budget 1.5 --confidence-threshold 0.35 \
  --trace-output request_plan.json
```

Query intent can be supplied as a token span, structured roles, or inferred structurally:

```
selector.select(prompt, query_spans=((q_start, q_end),))
selector.select(segments=[
    PromptSegment("query", question),
    PromptSegment("context", logs),
])
selector.select(prompt, policy="structural")
```

The selector returns half-open token intervals, confidence, score margin, method, and selected text. The model adapter can use those intervals to construct facets without assuming that query end equals prompt end. Structured segments are preferred in public applications; automatic inference does not override explicit metadata.

Manual mode uses the same profile representation and accepts explicit query-region, facet, root, neighbor, hop, conceptual-budget, K/V, threshold, layer, granularity, and materialization overrides. Overrides are request-local; they do not silently redefine the validated global ladder.

An independent implementation needs five invariants:

1. Separate conceptual and physical budgets in the public request and trace schema.
2. Reject evaluator labels before feature vectorization; do not merely promise not to use them.
3. Preserve prior selected identities and pages when escalating, and account for regenerated tokens separately.
4. Label every timing as component, end-to-end, measured, or extrapolated. A Python microbenchmark cannot become a production throughput claim by omission.
5. Keep query-role discovery separate from faceting, and preserve explicit caller spans or roles through retries unless the caller permits reinterpretation.

The relevant implementation is compact. `adaptive_runtime.py` defines profiles, feature validation, learned and hand controllers, stop policy, retries, consistency, and calibration. `query_regions.py` defines structured segments, explicit spans, structural discovery, multi-region selection, and cue reinterpretation. `effort_router.py` defines finite action spaces, independent categorical

heads, a cached semantic-input contract, and autoregressive heads. `factorized_control.py` defines valid independent actions, exact component accounting, token-weighted evidence metrics, Pareto dominance, allocation outcomes, and retry action names. `factorized_study.py` executes the frozen lattice and emits the oracle, router, and retry artifacts. `self_router.py` adds query-span pooling over tensors $[B, T, \dots]$, exact native Q/K capture from the layer-normalized residual stream, validation-only projection, grouped action decoding, prefill accounting, and Qwen prefix continuation. Q and K are reshaped to $[B, T, H, d_h]$ and passed through the layer’s own `q_norm/k_norm` before question-span pooling, matching Qwen’s pre-RoPE attention geometry rather than exposing raw projection outputs. The precompute script stores only pooled vectors; `self_router_study.py` owns oracle labels, grouped CV, target codebooks, and plots. Physical-runtime implementation remains behind the control/execution boundary.

14 Discussion

14.1 What adaptive control accomplished

The profile experiment first appeared to make HotpotQA uniformly high effort: 31 of 35 held-out units needed E2. Factorization changes that interpretation. Every Hotpot unit is solved at lower mean cost because most need only one facet, low K , and at most one hop; the expensive requirement is usually concentrated in roots or total parent budget. Coherent profiles are robust, but they pay for controls that an individual query does not require.

QASPER remains the more heterogeneous workload. Its factorized oracle uses $H = 0$ on 96.7% of held-out units and obtains a small quality gain over the profile lattice. These oracle results justify fine-grained actions in principle. The learned-router results deny the stronger claim that the present controller can allocate them safely: every R1–R3A variant loses quality through under-allocation. R0 therefore remains the supported initial controller despite measurable quantization regret.

Retry is similarly conditional. A bounded evaluator-side correction closes 19 of 24 initial failures, and eight corrections change only roots or facets. The remaining solvable failures need compound changes. This establishes a target for a future diagnostic classifier, not an achieved online correction policy. Universal cheap-first escalation remains inferior to direct prediction.

The add-on studies sharpen what escalation means. Failure may require a different cue rather than more roots under the same misplaced cue. Explicit roles and spans are both cheaper and more reliable than discovering role from raw serialization, so applications should expose them when possible. Automatic structural reinterpretation is a bounded fallback. Query-region breadth has the same two-sided character as other PRA controls: it can recover distributed instructions, but it can also dilute the objective and multiply facets, roots, and comparisons.

They also provide a controller-complexity stop. Independent and autoregressive heads now receive genuine factorized targets, yet neither improves the quality/cost frontier. The result is no longer an artifact of copying E0/E1/E2 labels. Self-encoding then removes the query-representation ambiguity without changing that result: validation-selected embedding-last and contextual hidden states both fall below observables on held-out units. The external-encoder gate therefore remains closed. A compact pretrained encoder is justified only after a larger grouped natural cohort establishes semantic headroom that this cohort does not.

14.2 Cognitive interpretation

The tested computational principle is modest:

encode task → allocate effort → execute → detect uncertainty → selective correction
→ stop.

Natural cognitive systems plausibly vary retrieval cues and deliberation under uncertainty, but the present controller is not a biological model. Its scientific content is the measured frontier between quality and expected resource use.

14.3 Program handoff

Paper 3.5 ends at an adaptive initial action, observable sufficiency evidence, and one bounded correction. Kernel compilation, indexed-search crossover, physical memory hierarchy, batching, and concurrency are Paper 5.5 questions. First-class vLLM logical blocks, asynchronous memory movement, and scheduler awareness are Paper 6 questions. This boundary keeps control quality from being conflated with a Python prototype’s throughput.

15 Limitations

The adaptive quality endpoint is evidence-chain completion from frozen traces, not a live generated answer. Output confidence comes from a separate controlled model, so a combined routing/output controller remains untested. The held-out adaptive cohort contains only 65 example-seed units and inherits Paper-2.5’s model and dataset choices.

The supported learned controller is the R0 profile selector. Factorized R1–R3A use independently measured targets, but the cohort is too small to establish reliable natural parameter interactions. R2 uses actual questions through a dependency-free signed-hash encoder rather than a pretrained NLP encoder. The output calibration cohort is small and imbalanced. Semantic consistency uses token F1 and can miss paraphrase and contradiction.

The self-router sweep contains 32 unique questions from HotpotQA and QASPER and one frozen Qwen3-0.6B backbone. Repeated model seeds provide routing variation but not additional linguistic identities. Validation-selected embedding-last does not generalize to held-out units, while the numerical held-out maximum for embedding mean was not validation-selected and is not a confirmed gain. The contextual condition processes an ordinary truncated source prompt as a diagnostic; it is not the normal PRA execution path. 2Wiki, MuSiQue, learned attention pooling, another model family, and an external encoder remain untested because they lack this add-on’s frozen factorized surface or fail the staged expansion rule. CUDA timings are from one GTX 950M and are not deployment latency claims.

Query-region evaluation remains deterministic. Semi-natural email, code, log, URL, tool-output, and conversation fixtures broaden surface form but are not sampled from a natural prompt distribution. The adversarial structural failure shows that ambiguous role resolution remains open. Tokenizer-specific span mapping and a lightweight learned span scorer remain untested.

The factorized endpoint is frozen-score evidence-chain recovery, not generated-answer quality. The original factorized surface ranks existing cached facets. Section 9 adds parameter-free facet construction from frozen query states, but it is not a trained semantic interpreter. Its first 45/29 analysis is post-hoc within the prior test cohort; the subsequent full-surface callback study restores the original 58/74 partition. Search-order admission does not exploit $B_s > B_{kv}$ in any held-out minimum-cost oracle. The targeted correction audit uses evaluator success to choose an action and must not be read as an oracle-free retry result. Five QASPER units remain unsolved throughout the measured lattice.

The search-method extension inherits Paper 2.6’s small matched-budget discovery corpus and exhaustive Python implementations. It contains 74 held-out root identities but only 68 with successor labels; complete broad-span recovery is zero at the four-chunk request budget. The per-example method oracle uses evaluator labels, and the 32-identity factorized join is a composition of two measured surfaces, not a joint execution. Leave-one-dataset-out method accuracy is weak. The useful-address gate abstains on every held-out example, and the learned retry action accuracy is .221. The self-query method comparison is four-fold out-of-fold because all overlapping cache identities belong to the Paper-2.6 test split; it is not a validation-frozen downstream result. Query-region $\times$ method interaction is unidentifiable because only two examples use the minority layout. Unlike method operations are intentionally not collapsed into one systems-cost scalar.

The first graph-facet replay measures root discovery only and uses a post-hoc 45/29 split. The newer 29,700-action study repairs both limitations for one fixed two-root/two-successor budget, but it does not cross hop depth, effort, per-facet routing, or variable physical admission. Hierarchical overlap counts coarse parent/child reuse; global hard-community overlap is zero by construction. Prototype graph timings include Python control flow and are not serving measurements. The single callback is tested with small ridge and threshold controllers over 58 validation identities; B7 is an evaluator oracle, not a deployable policy. The eight-example decode cohort verifies native-K/V propagation but is too small for dataset-level answer-quality claims.

Finally, no full PRA backbone is trained here. Learned consumer adaptation and full-weight scaling are deferred to Paper 4.

16 Conclusion

PRA effort is better represented by separable interpretation, search, and admission controls than by one diagonal profile lattice. At matched complete-chain quality, factorization reduces mean held-out HotpotQA cost from 34.18 to 19.98 and matched QASPER cost from 16.11 to 8.95. The result resolves the main scientific question: profile quantization hides useful low-cost combinations.

The control problem is not solved. R1–R3A reduce nominal cost by under-allocating and remain below R0 quality, even with factorized targets. Signed hashing, deep self-state, native Q/K, and the validation-selected contextual upper bound do not reverse that result. R0 stays the SDK default. One bounded oracle corrective action exposes substantial retry headroom, but compound changes and five unsolved QASPER units prevent a stronger online claim.

Candidate normalization explains where that headroom comes from. QASPER’s factorized oracle reaches .917 evidence-parent recall and .796 precision while admitting 10.9% of candidates, compared with .750, .450, and 18.2% for R0. Retry reaches the oracle recall but admits 19.0% and has lower precision. HotpotQA retry raises recall to .948 at a denser 75.6% admission fraction. Thus adaptive control can expose sparse working sets, especially on QASPER, but the learned controller has not yet found them reliably.

Retrieval representation adds another genuine but unresolved control surface. Exact root search selected on validation reaches .066 recall, while the root oracle reaches .104. At the two-stage budget, the fixed BM25 $\rightarrow$ native-semantic pair reaches .030, the interpretable cascade .048, the learned joint selector .032, and the joint method oracle .143. Root and successor methods should therefore remain independent actions, but a validation-fixed pair remains the conservative default until selection improves. C0 is the strongest measured deployable cascade, not a universal policy: it loses to the fixed pair on QASPER and MuSiQue. Always switching methods hurts; learned targeted retry raises recall only from .032 to .037.

Graph faceting completes the capability side of that baseline without solving prediction. The

exploratory post-hoc split shows per-facet headroom, but the stronger full-surface result is conservative: B1 and B2 both reach about .180 recall, while B3–B5 reach .167–.168. The transparent threshold policy reaches .188 but does not beat the .205 validation-fixed global policy. B6 and B7 are identical at .365, so this action surface provides no stage-wise oracle gain. The callback API is useful as an experimental boundary and trace format, not as the SDK default. A future callback iteration needs different supervision, state, or legal actions; more capacity on the same labels is not the next justified step.

Adaptive PRA must also decide what the query is. Explicit roles remain preferred; structural and recent interpretations should compete under confidence because either can fail under serialization or decoys. The contribution is therefore a measured control surface, a conservative default, and a bounded retry endpoint. The runtime now follows the six-stage cycle in Equation 16, followed by evaluation and targeted retry. A self-prefill captured before layer 24 can be reused exactly, which makes future semantic control cheaper, but the present controller does not earn that added complexity. Making those decisions fast belongs to Paper 5.5; making them native to vLLM belongs to Paper 6.

A Artifact Manifest

All normalized artifacts are under `docs/papers/shared/results/paper3_5.adaptive_pra`. They include:

- effort profiles, controller features, training and calibration, oracle ceiling, regret, retry traces and outcomes, and the adaptive frontier;
- query-region configurations plus per-layout, detection, retrieval, output, displacement, facet, root, retry, session, semi-natural, and adversarial summaries, findings, and figures;
- router profiles, action spaces, oracle targets, R0–R3 results, per-head calibration, interaction diagnostics, regret, latency, query-class summaries, findings, and frontier figure;
- factorized action space, per-unit oracles and Pareto fronts, quantization regret, parameter requirements/interactions, precision–recall and search–admission surfaces, factorized router targets/results, bounded retry rows, findings JSON, and figures;
- normalized controller rows, candidate and evidence counts, search/admission fractions, physical native-K/V tokens, complete-recovery and Pareto tables, allocation outcomes, claim audit, and ten normalized figures;
- self-router representation/configuration manifests, layer/pooling/profile sweeps, contextual drift, feature ablation, target-family rows, under/over-allocation, exact-reuse accounting, findings, and figures;
- adaptive-search action specification, root/successor oracles, transition matrix, fixed/cascade/learned policy rows, leave-one-dataset-out and self-query controls, useful-address and retry rows, factorized composition, operation-specific cost accounting, claim audit, findings, and ten figures;
- request/reply natural replay rows for five facet modes, per-facet method outcomes, graph calls, density, overlap, pairwise work and latency, five-seed B1–B4 controllers, B7 oracle, scope audit, findings JSON, and two figures;

- the complete 29,700-row request/reply surface, 5,940 persisted root states, five-seed standard and leave-one-dataset-out one-shot/callback rows, threshold and feature ablations, graph accounting, native-K/V decode rows, findings manifests, and callback figures;
- inherited systems and matched-proxy rows retained for Paper-5.5 provenance, not counted as Paper-3.5 contributions.

The complete study runs with:

```
python -m experiments.paper3_5_adaptive_pra.run_study
python -m experiments.paper3_5_adaptive_pra.factorized_study
python -m experiments.paper3_5_adaptive_pra.normalized_efficiency
python -m experiments.paper3_5_adaptive_pra.precompute_self_router_features --device cuda
python -m experiments.paper3_5_adaptive_pra.self_router_study --device cuda
python -m experiments.paper3_5_adaptive_pra.adaptive_search_methods
python -m experiments.paper2_7_query_graph.run_natural_retrieval --device cuda
python -m experiments.paper3_5_adaptive_pra.request_reply_graph_study
python -m experiments.paper3_5_adaptive_pra.precompute_request_reply_query_states --device cuda
python -m experiments.paper3_5_adaptive_pra.request_reply_full_surface --device cuda
python -m experiments.paper3_5_adaptive_pra.root_callback_study
python -m experiments.paper3_5_adaptive_pra.request_reply_end_to_end --device cuda
python -m pytest
```

B Decision and Accounting Details

The learned controller uses standardized ridge least squares against one-hot E0/E1/E2 targets. The failure controller uses the same estimator with success/failure targets. Validation selects a .3 maximum predicted-failure threshold for retry stopping. AUROC is computed by exact pairwise ranking, AUPRC by average precision over descending risk, ECE with ten equal-width bins, and Brier score as mean squared probability error.

Risk-coverage sorts examples from lowest to highest predicted failure. At coverage c , selective risk is the failure fraction among the lowest-risk cN observations. We retain every point in the calibration JSON rather than reading values back from a rendered figure.

Search comparisons count query-key dot products. K/V bytes count both keys and values. Page fragmentation is allocated page slots minus valid tokens. Batch allocation counts padded or page rounded token slots. CPU benchmark bytes are physical tensor sizes; HBM/request is an explicitly marked extrapolation from token count, dimension, dtype, and K/V multiplicity.

For factorized control, profile quantization regret is defined only when both the profile and factorized action spaces contain a complete-chain configuration:

$$R_{\text{quantization}} = C_{\text{profile oracle}} - C_{\text{factorized oracle}}. \quad (22)$$

Under-allocation means the selected action misses a chain recovered by the factorized oracle. Over-allocation means it matches oracle quality at higher cost. A failed unit is never counted as a cost saving. Held-out oracle labels are used only for evaluation and corrective-action upper bounds.

For self-routing, each grouped-CV fold fits centering and PCA components on unique training identities only, then applies the frozen transform to its validation identities. Final held-out models refit on all validation identities. Query-prefill cost is always added to the selected retrieval action unless the exact prefix state is reused. Parameter-label accuracy is diagnostic; Tables 16 and 17 report downstream chain quality, under-allocation, and total cost instead. The source-plus-query condition pays all 256 ordinary context tokens and is never presented as a free in-flight signal.

C Representative Public Trace

```
{
  "attempt": 1,
  "effort": "E1_medium",
  "control_vector": {
    "F": {"policy": "syntactic", "count": 2},
    "R": 2, "K": 4, "H": 1,
    "B": {"conceptual_parents": 4, "native_kv_tokens": 512},
    "theta": 0.45,
    "L": {"search": [24, 27], "consumer": [26, 27]},
    "G": 128, "M": "evidence_centered"
  },
  "active_native_kv": 512,
  "incorrect_probability": 0.28,
  "retry_consistency": 0.83,
  "reused_search_items": 1,
  "reused_kv_tokens": 256,
  "stop_reason": "confidence"
}
```

This trace is a runtime decision record. It gives an application enough information to display effort and cost or request a manual retry, without exposing hidden reasoning.

References

- [1] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2025.
- [2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [3] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.
- [4] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.
- [5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: <https://arxiv.org/abs/2309.06180>.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [7] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: <https://arxiv.org/abs/2404.14469>.
- [8] Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. ClusterKV: Manipulating LLM KV cache in semantic space for recallable compression. *arXiv preprint arXiv:2412.03213*, 2024.

- [9] Amirkeivan Mohtashami and Martin Jaggi. Landmark attention: Random-access infinite context length for transformers. *arXiv preprint arXiv:2305.16300*, 2023.
- [10] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143*, 2024.
- [11] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.
- [12] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021. Preprint: <https://arxiv.org/abs/2003.05997>.
- [13] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 10014–10037, 2023.
- [14] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.
- [15] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. StreamingLLM: Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.
- [16] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.
- [17] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: <https://openreview.net/forum?id=RkRrPp7GK0>.